

Matteo Palmonari

matteo.palmonari@unimib.it

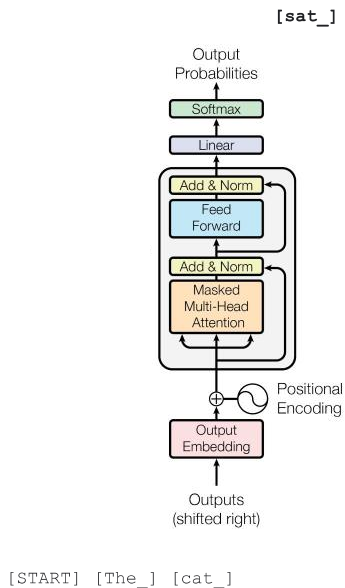
Transformers and Language Models



Transformers

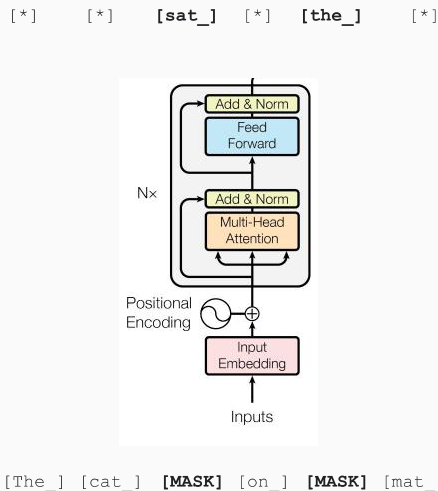
Architectures and key components

Decoder-only GPT



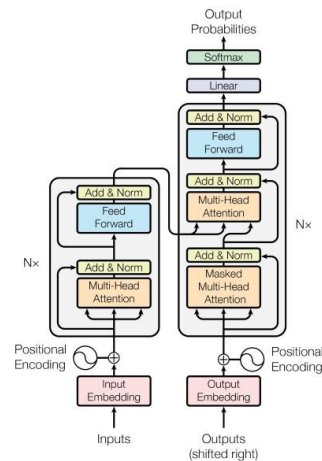
Transformer image source: "Attention Is All You Need" paper

Encoder-only BERT



Enc-Dec T5

Das ist gut.
A storm in Attala caused 6 victims.
This is not toxic.



Translate EN-DE: This is good.
Summarize: state authorities dispatched..
Is this toxic: You look beautiful today!

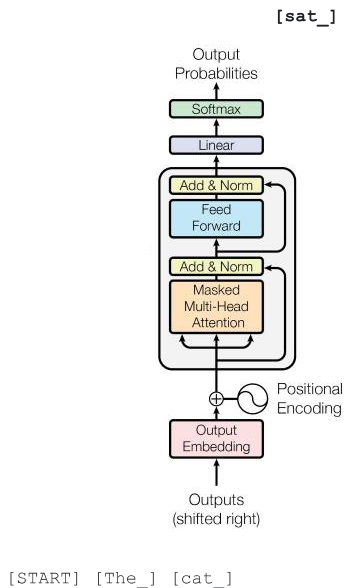
Transformers

Architectures and key components

Key Modules in Transformers

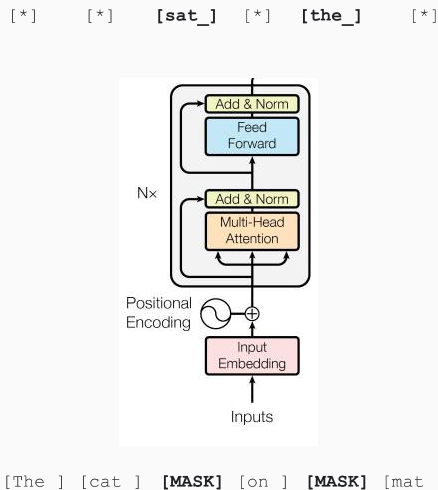
Encoders: Natural Language Understanding

Decoder-only GPT



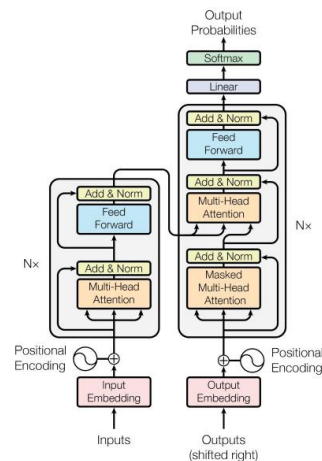
Transformer image source: "Attention Is All You Need" paper

Encoder-only BERT



Enc-Dec T5

Das ist gut.
A storm in Attala caused 6 victims.
This is not toxic.

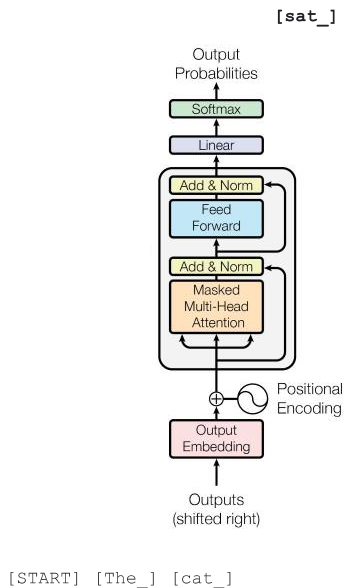


Translate EN-DE: This is good.
Summarize: state authorities dispatched..
Is this toxic: You look beautiful today!

Some Remarks on this Presentation

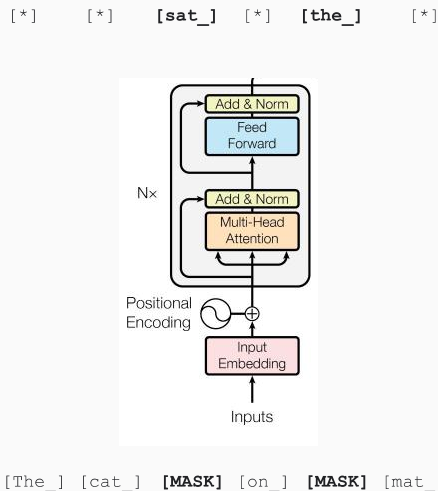
- I'm trying to balance: more details on some key topics (e.g., attention), less details on other topics (e.g., feed forward networks or training strategies)
- However, sometimes I keep slides with more details even if in the presentation I do not discuss all the details
 - The presentation may have more details than what I expect you to learn; i.e., the test will not contain questions on details that are not explicitly discussed during the lesson
 - You may come back to more technical slides one day, when you will be more familiar
- I will teach key aspects of transformers using encoder transformers and BERT
- I will add something to explain decoder transformers, i.e., autoregressive generation
- I'll mostly skip encoder-decoder transformers

Decoder-only GPT



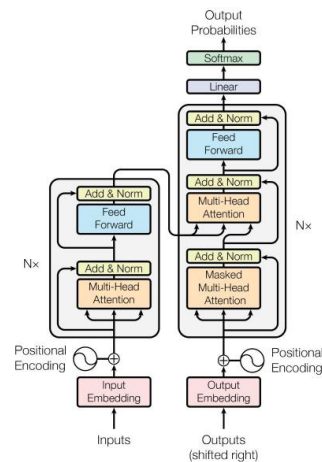
Transformer image source: "Attention Is All You Need" paper

Encoder-only BERT



Enc-Dec T5

Das ist gut.
A storm in Attala caused 6 victims.
This is not toxic.

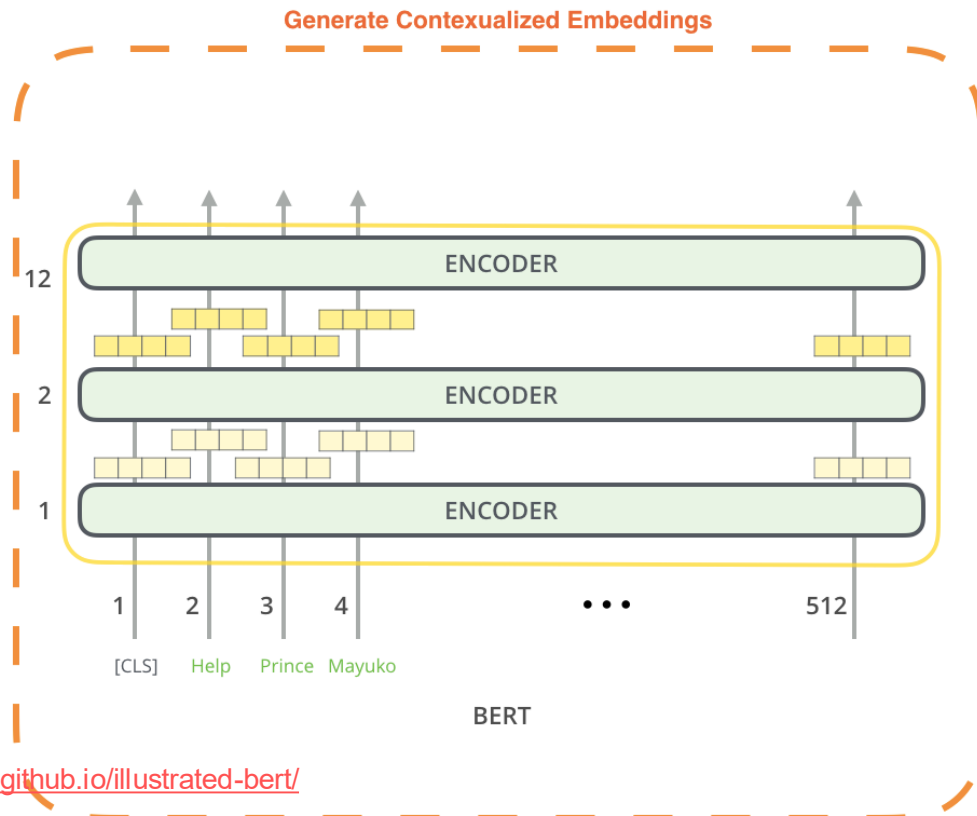


Translate EN-DE: This is good.
Summarize: state authorities dispatched...
Is this toxic: You look beautiful today!

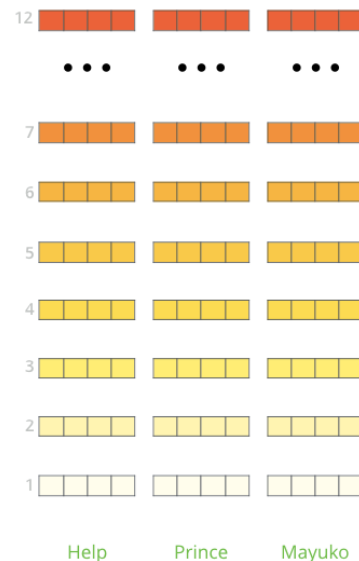
Transformers: Sequence Transformations

Learn
how to
convert
an input
sentence
to vectors

Several
layers



The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

Sequence of what? Input to BERT and tokenization

Sentence = "I am a policewoman"

Is tokenized as:

- ['**[CLS]**¹', 'I', 'am', 'a', 'police', '**##woman**²', '**[SEP]**¹']

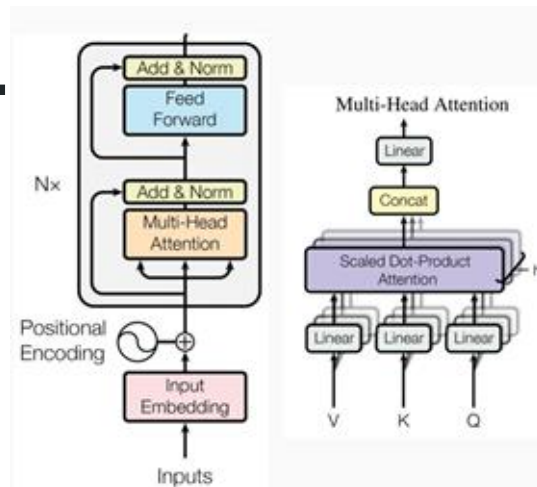
Remark: different models use different tokenizers!
All LMs are based on tokenization

Things to remember:

1. [CLS] and [SEP] are added to the start and to the end of the sentence¹
2. BERT uses a very specific tokenizer
 - Vocabulary size (EN) $\sim 30k$ ($\ll$ word lexicon)
3. The input has a fixed number of tokens, e.g., 512 in BERT $\rightarrow$ "context window"

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features
- Layer norm and residuals
 - Healthy training
- Positional embeddings
 - Relative positioning learning



Transformers



LSTMs

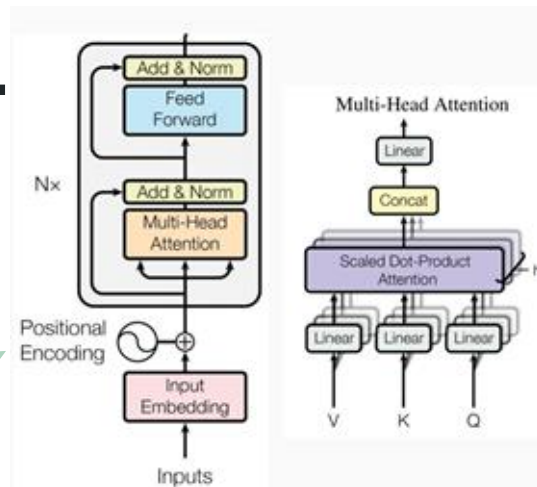
<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features
- Layer norm and residuals
 - Healthy training
- **Positional embeddings**
 - Relative positioning learning

Each token embedding is transformed to reflect its position



Transformers



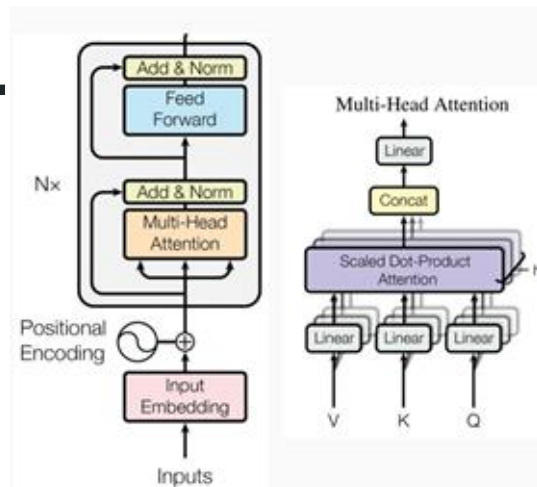
LSTMs

<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features
- Layer norm and residuals
 - Healthy training
- Positional embeddings
 - Relative positioning learning



Transformers



LSTMs

<https://jalammar.github.io/illustrated-transformer/>

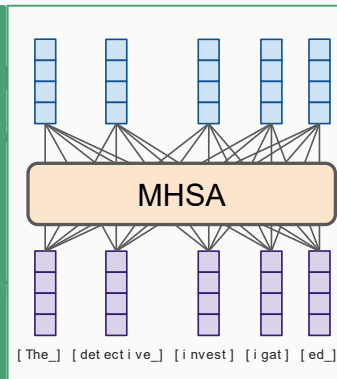
<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

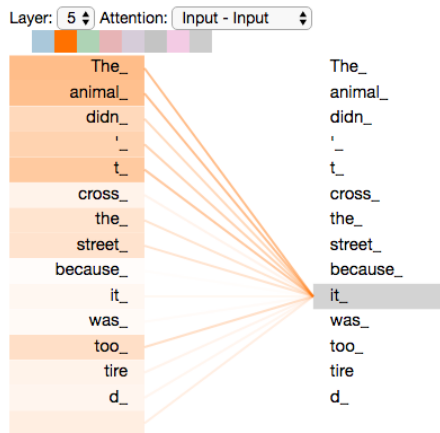
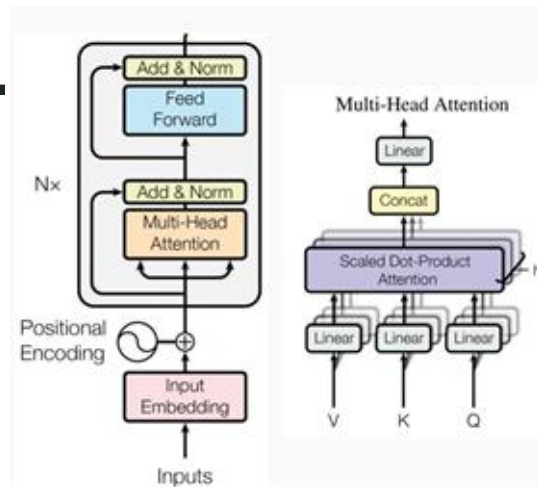
- Multi-headed self attention
 - Context modeling

Each token can "look around" the whole input, and decide how to **update its representation** based on what it sees.

The input sequence is used to create queries, keys, and values



From: Lucas Beyer [...]



As we are encoding the word "it" in encoder #5 (the top encoder in the stack), part of the attention mechanism was focusing on "The Animal", and baked a part of its representation into the encoding of "it".

<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

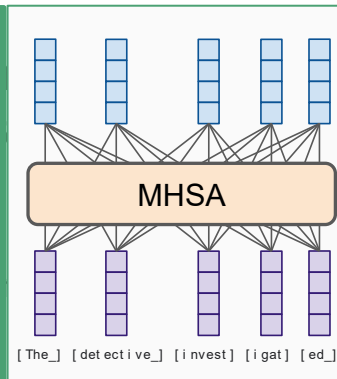
Encoder Transformer - BERT

- Multi-headed **self** attention

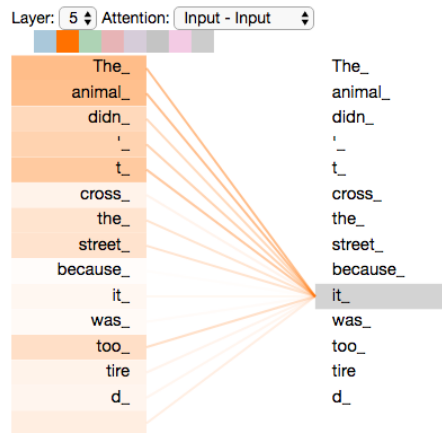
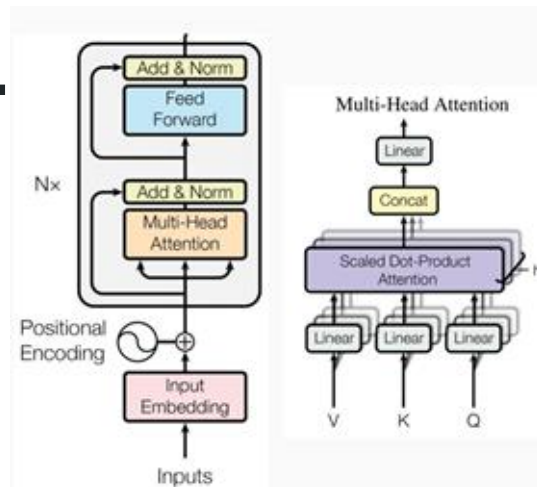
- Context modeling

Each token can "look around" the whole input, and decide how to **update its representation** based on what it sees.

The input sequence is used to create queries, keys, and values



From: Lucas Beyer [...]



As we are encoding the word "it" in encoder #5 (the top encoder in the stack), part of the attention mechanism was focusing on "The Animal", and baked a part of its representation into the encoding of "it".

<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

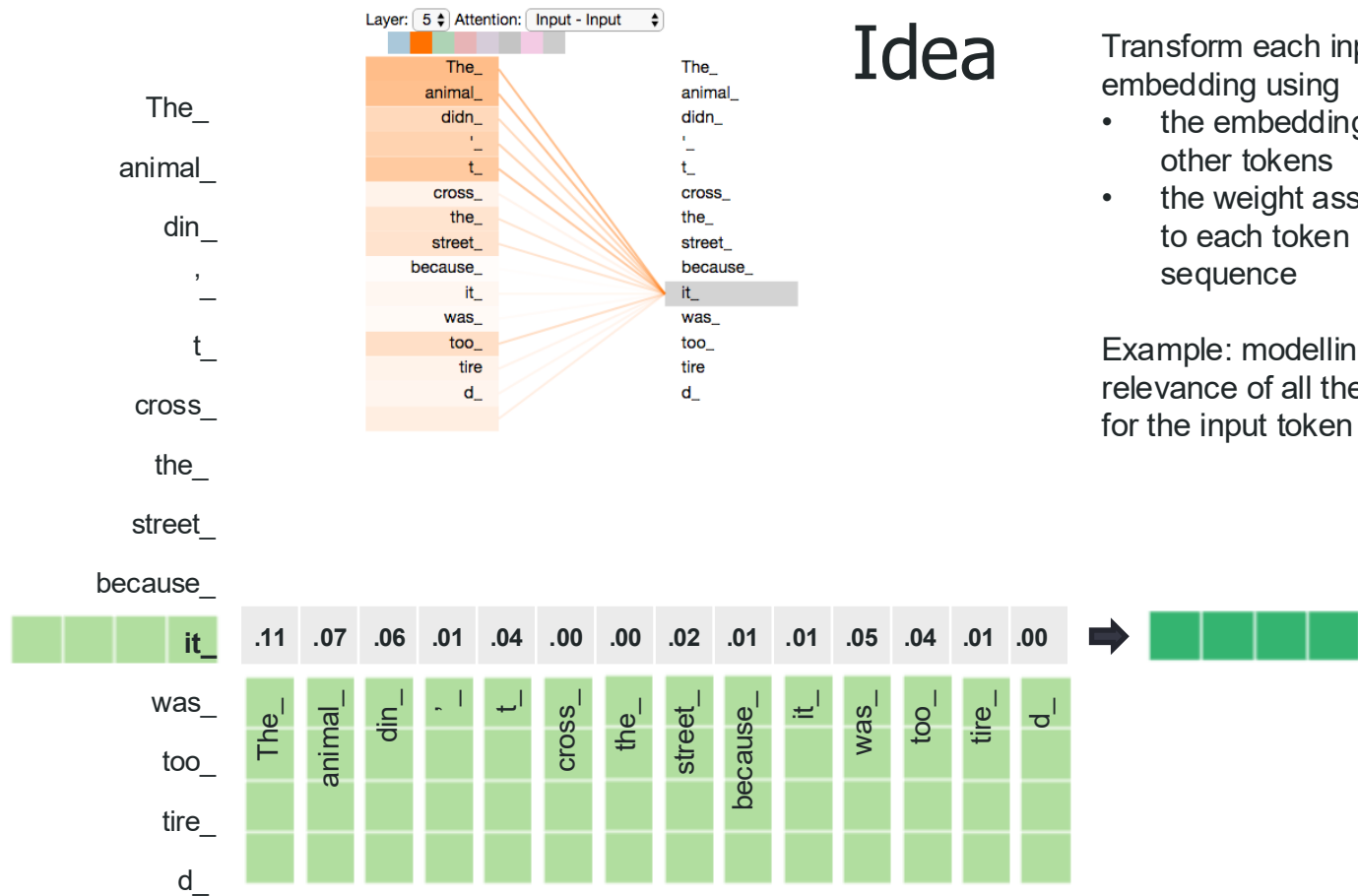
The animal didn't cross the street because it was too tired

The Attention Idea

Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

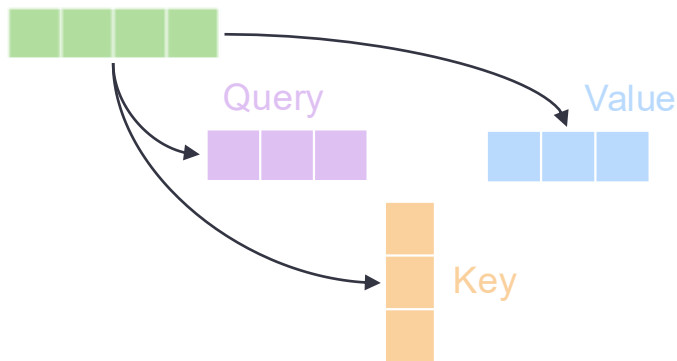


The animal didn't cross the street because it was too tired

The Attention

Idea

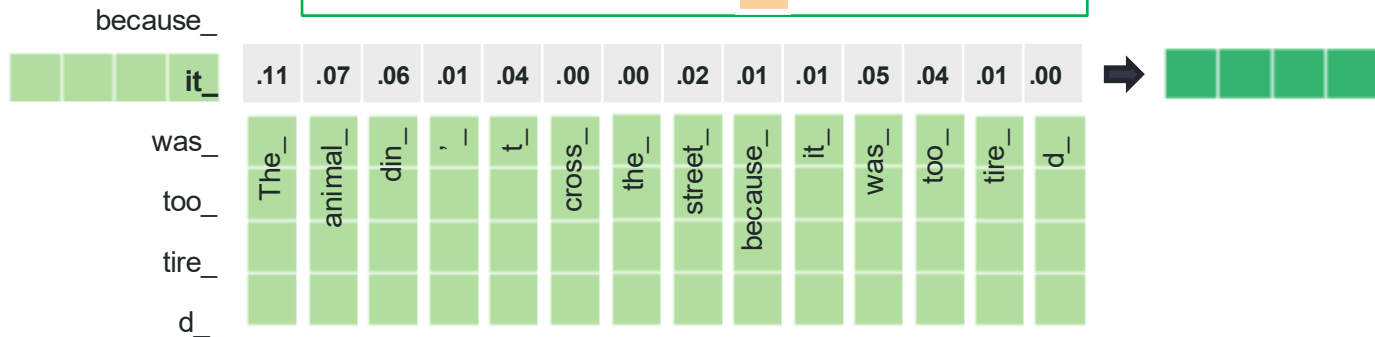
Trick: map each input embeddings to three smaller “attention” vectors and use these vectors instead of original vectors



Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

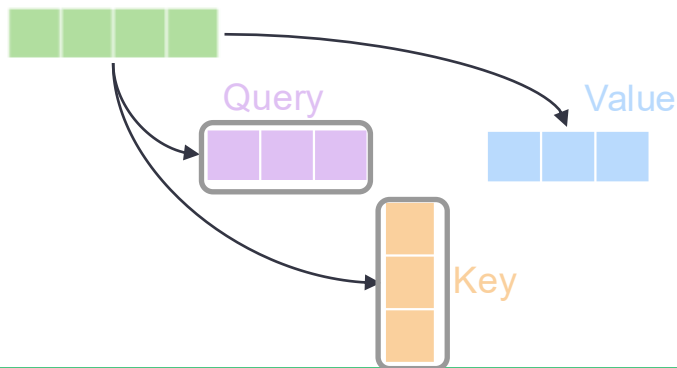


The animal didn't cross the street because it was too tired

The Attention

Idea

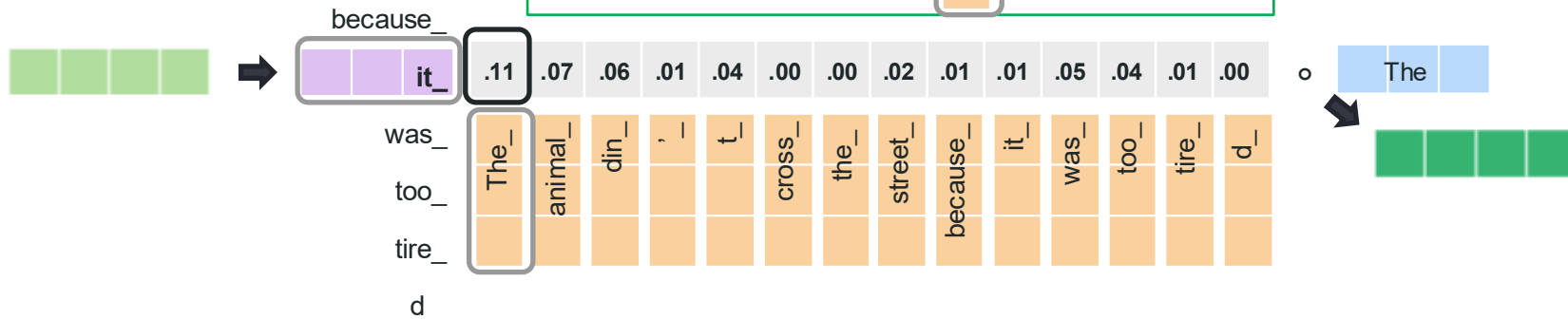
Trick: map each input embeddings to three smaller “attention” vectors and use these vectors instead of original vectors



Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

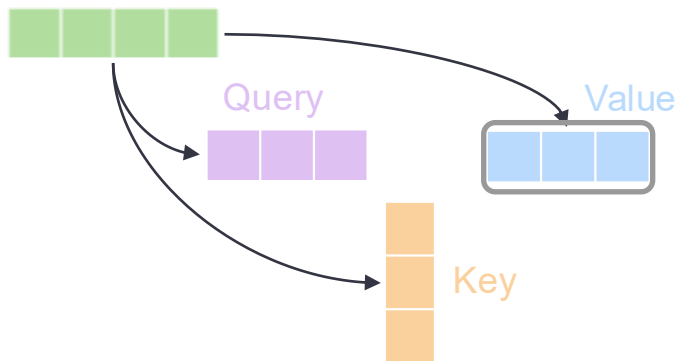


The animal didn't cross the street because it was too tired

The Attention

Idea

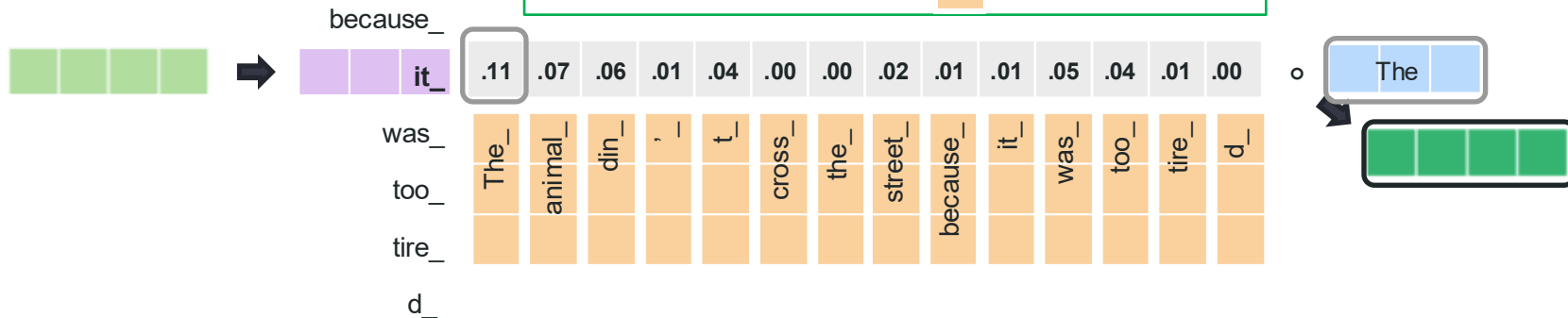
Trick: map each input embeddings to three smaller “attention” vectors and use these vectors instead of original vectors



Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”



Attention

Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

Compute everything in parallel for all input tokens/embeddings by using matrix multiplication

The_														
animal_														
din_														
'_														
t_														
cross_														
the_														
street_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
because_	.00	.00	.01	.03	.02	.15	.06	.11	.01	.08	.09	.03	.11	.03
it_	.11	.07	.06	.01	.04	.00	.00	.02	.01	.01	.05	.04	.01	.00
was_	.01	.01	.03	.01	.04	.00	.00	.01	.01	.15	.01	.07	.18	.11
too_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
tire_														
d_														
	The_	animal_	din_	'_	t_	cross_	the_	street_	because_	it_	was_	too_	tire_	d_

Query
vectors

The_														
animal_														
din_														
'_														
t_														
cross_														
the_														
street_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
because_	.00	.00	.01	.03	.02	.15	.06	.11	.01	.08	.09	.03	.11	.03
it_	.11	.07	.06	.01	.04	.00	.00	.02	.01	.01	.05	.04	.01	.00
was_	.01	.01	.03	.01	.04	.00	.00	.01	.01	.15	.01	.07	.18	.11
too_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
tire_														
d_														
Key vectors	The_	animal_	din_	'_	t_	cross_	the_	street_	because_	it_	was_	too_	tire_	d_

Attention

Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

Compute everything in parallel for all input tokens/embeddings by using matrix multiplication

Value
vectors

The_
animal_
din_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire_
d_

Query
vectors

The_
animal_
din_
,_
t_
cross_
the_
street_
because_
it_
was_
too_
tire_
d_

Key
vectors

The_
animal_
din_
,_
t_
cross_
the_
street_
because_
it_
was_
too_
tire_
d_

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

= Z

The formula: matricial calculation (explained in next slides)

...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
.00	.00	.01	.03	.02	.15	.06	.11	.01	.08	.09	.03	.11	.03	
.11	.07	.06	.01	.04	.00	.00	.02	.01	.01	.05	.04	.01	.00	
.01	.01	.03	.01	.04	.00	.00	.01	.01	.15	.01	.07	.18	.11	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	

Attention

Transform each input token embedding using

- the embedding of the other tokens
- the weight associated to each token in the sequence

Example: modelling the relevance of all the tokens for the input token “it”

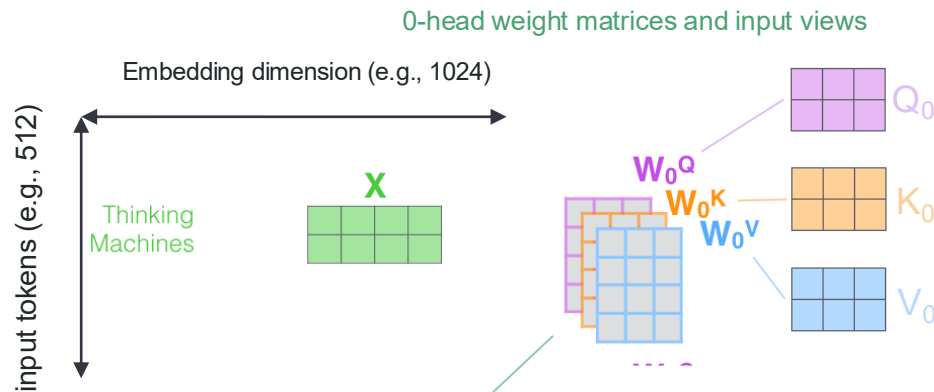
Compute everything in parallel for all input tokens/embeddings by using matrix multiplication

Value
vectors

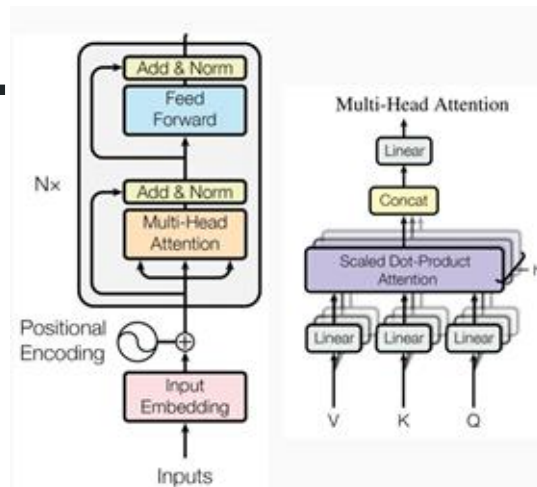
The_
animal_
din_
,_
t_
cross_
the_
street_
because_
it_
was_
too_
tire_
d_

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling



The Q, K, V values for the input are computed using weights (projection matrices) learned at training time



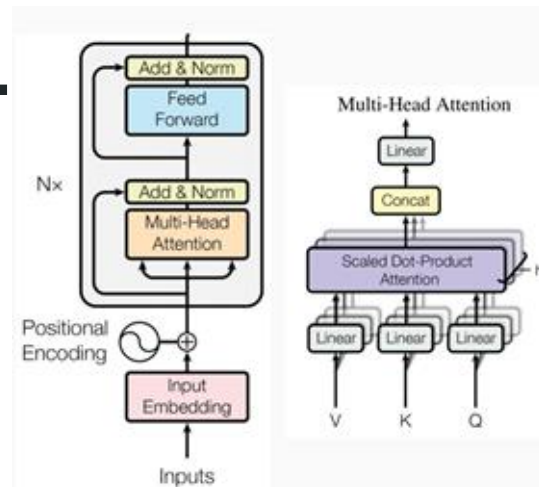
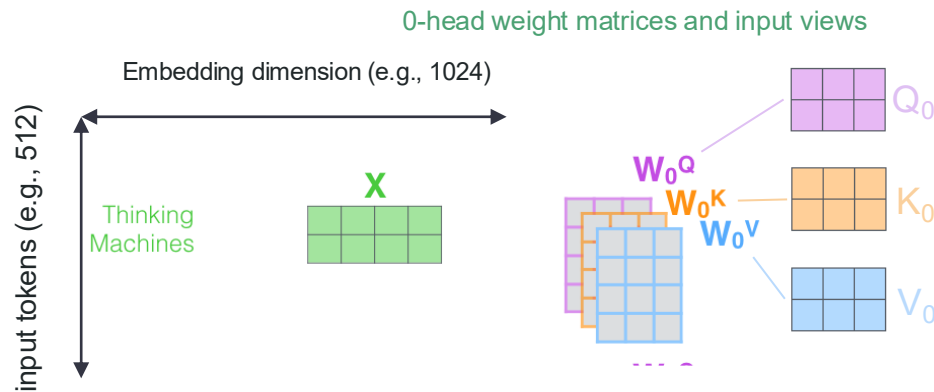
$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$
$$= Z$$

<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

Z

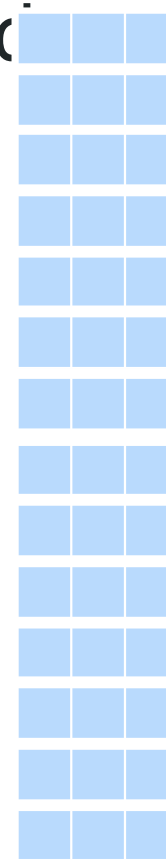
$\begin{bmatrix} .02 & .08 \\ .06 & .04 \end{bmatrix}$

<https://jalamar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

The Attention

Input



Transform each input token embedding based on

- the embedding of the other tokens
- the weight associated to each token in the sequence

Modelling the relevance of all the tokens for the input token "it"

Compute everything in parallel for all input tokens/embeddings by using matrix multiplication

Value vectors

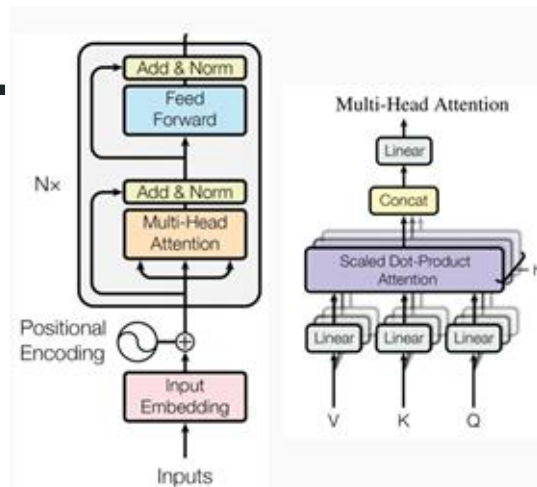
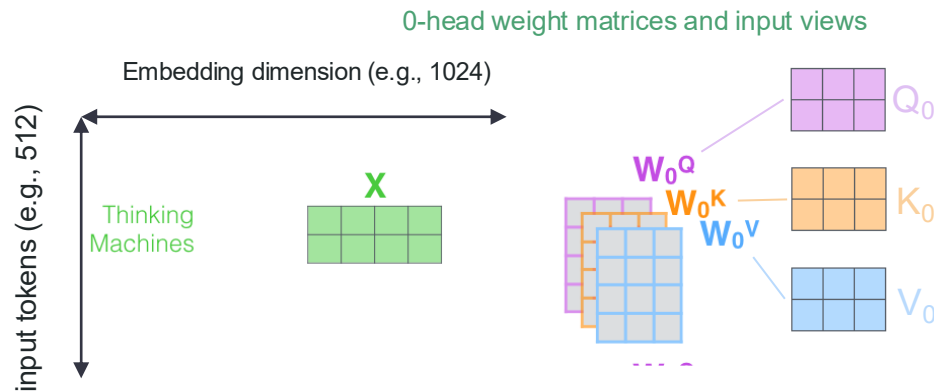
Query vectors

softmax

The_														
animal_														
din_														
'_														
t_														
cross_														
the_														
street_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
because_	.00	.00	.01	.03	.02	.15	.06	.11	.01	.08	.09	.03	.11	.03
it_	.11	.07	.06	.01	.04	.00	.00	.02	.01	.01	.05	.04	.01	.00
was_	.01	.01	.03	.01	.04	.00	.00	.01	.01	.15	.01	.07	.18	.11
too_	...	...	...	...	...	...	...	...	...	...	...	...	...	...
tire_														
d_														
Key vectors	The_	animal_	din_	'_	t_	cross_	the_	street_	because_	it_	was_	too_	tire_	d_

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

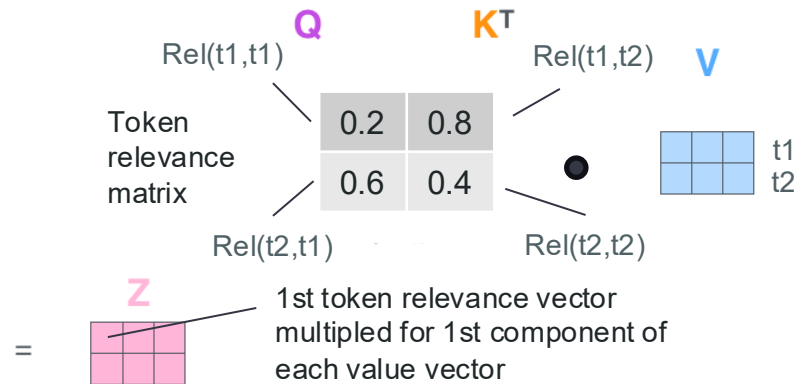
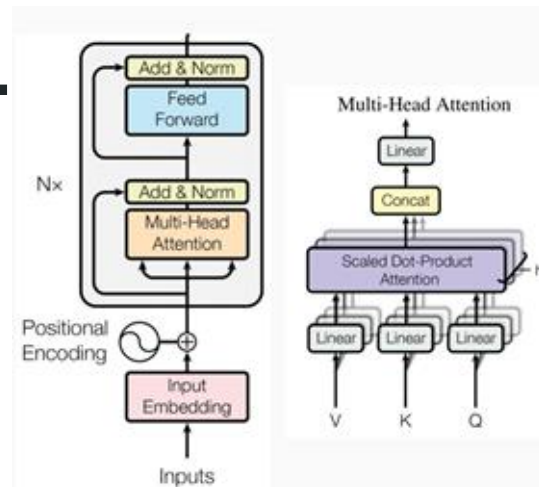
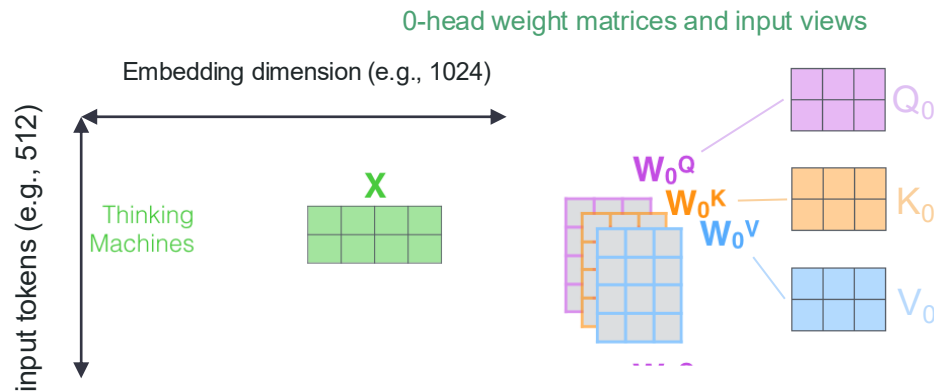
For large values of d_k , the dot product grows large in magnitude, pushing the softmax function into regions where it has extremely small gradients

<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling

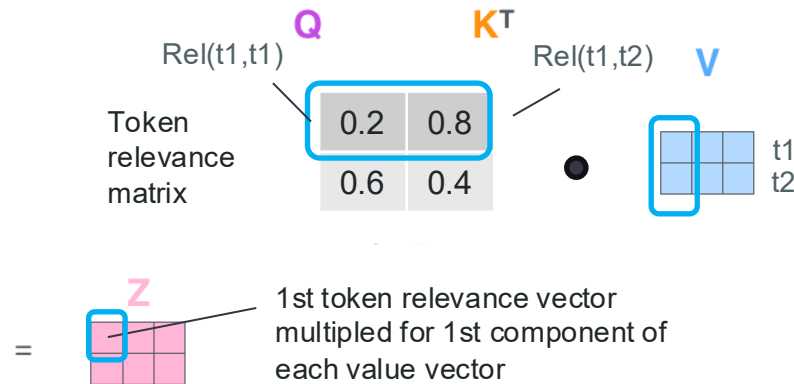
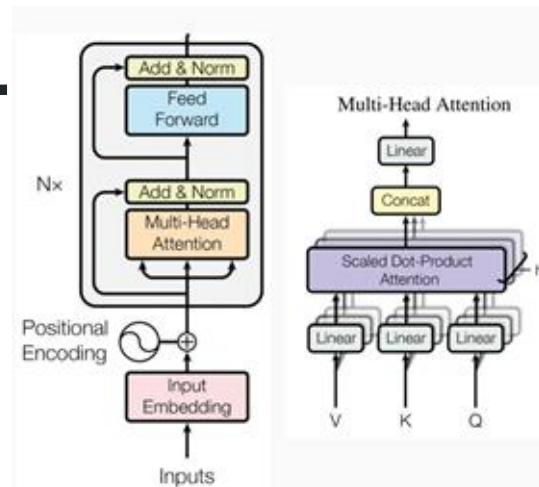
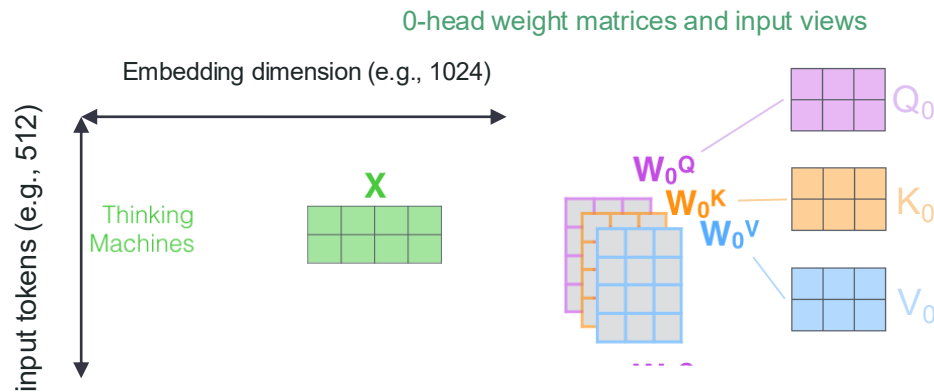


<https://jalamar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling

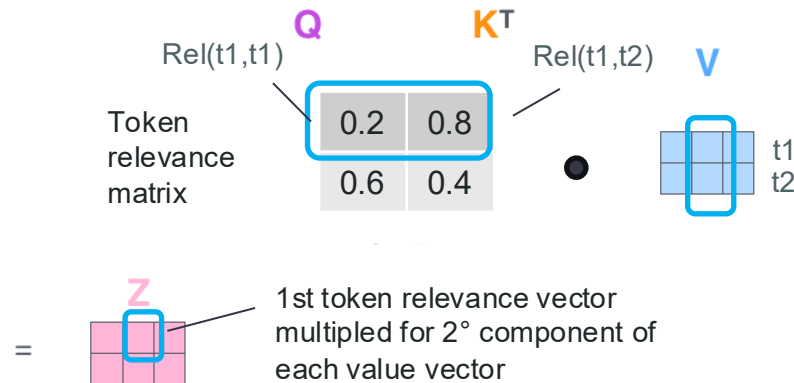
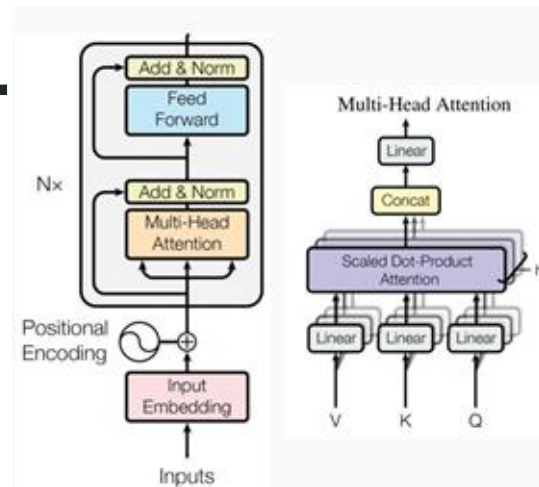
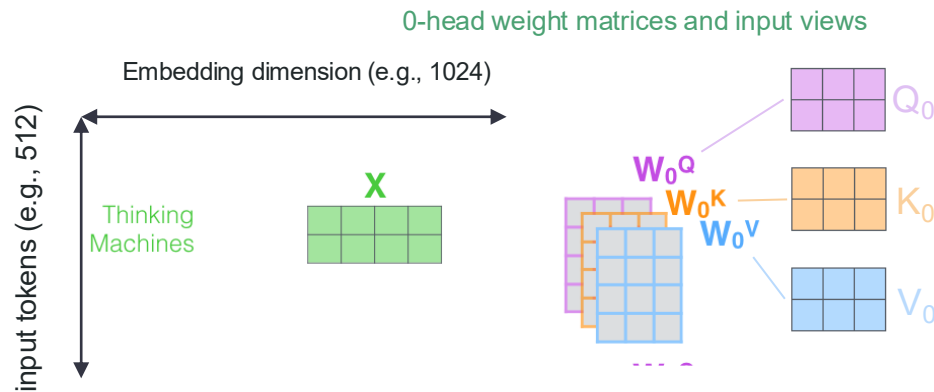


<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling

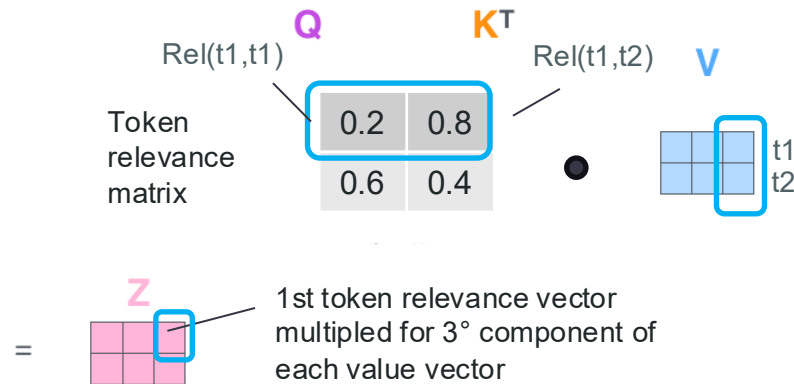
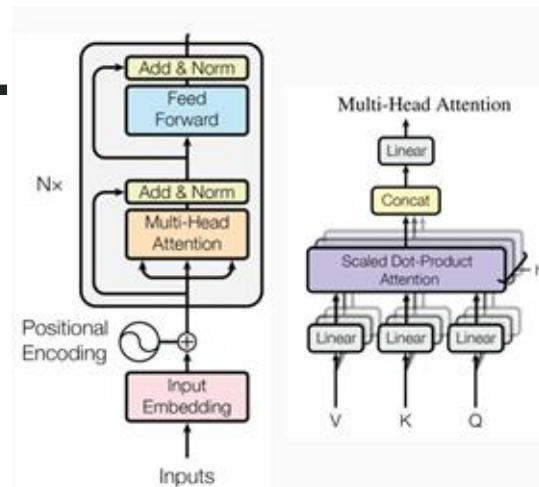
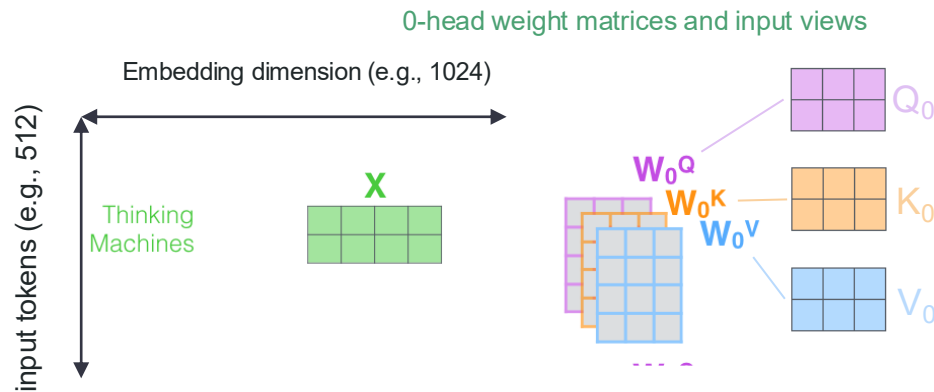


<https://jalamar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling



<https://jalamar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

1) This is our input sentence*

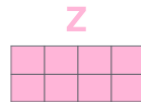
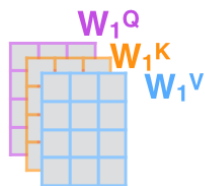
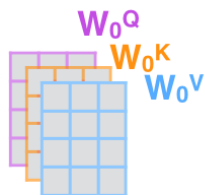
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

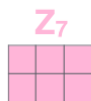
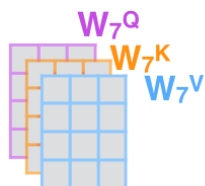
Thinking Machines



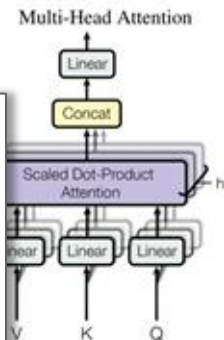
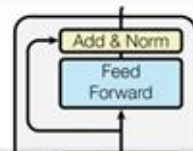
...

...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



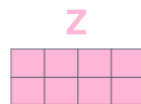
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

Encoder Transformer - BERT

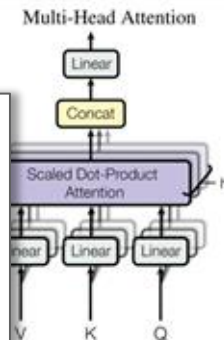
- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply **X** or **R** with weight matrices
- 4) Calculate attention using the resulting **Q/K/V** matrices
- 5) Concatenate the resulting **Z** matrices, then multiply with weight matrix **W^O** to produce the output of the layer



W^O



Remember:
this is the
desired
embedding
dimension



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

Diagram illustrating the attention mechanism formula, showing the calculation of the attention weights and the resulting output Z .

Encoder Transformer - BERT

1) This is our input sentence*

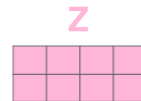
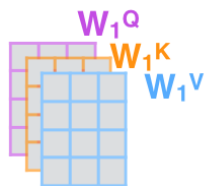
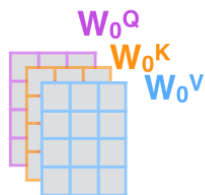
2) We embed each word*

3) Split into 8 heads. We multiply **X** or **R** with weight matrices

4) Calculate attention using the resulting **Q/K/V** matrices

5) Concatenate the resulting **Z** matrices, then multiply with weight matrix **W^O** to produce the output of the layer

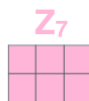
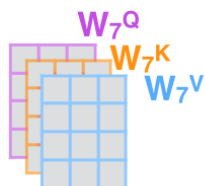
Thinking Machines



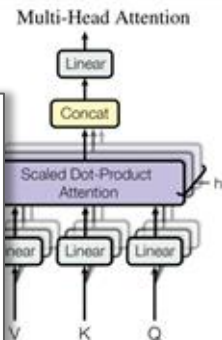
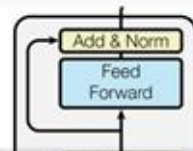
...

...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



$$\text{softmax} \left(\frac{\mathbf{Q} \times \mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

$$= \mathbf{Z}$$

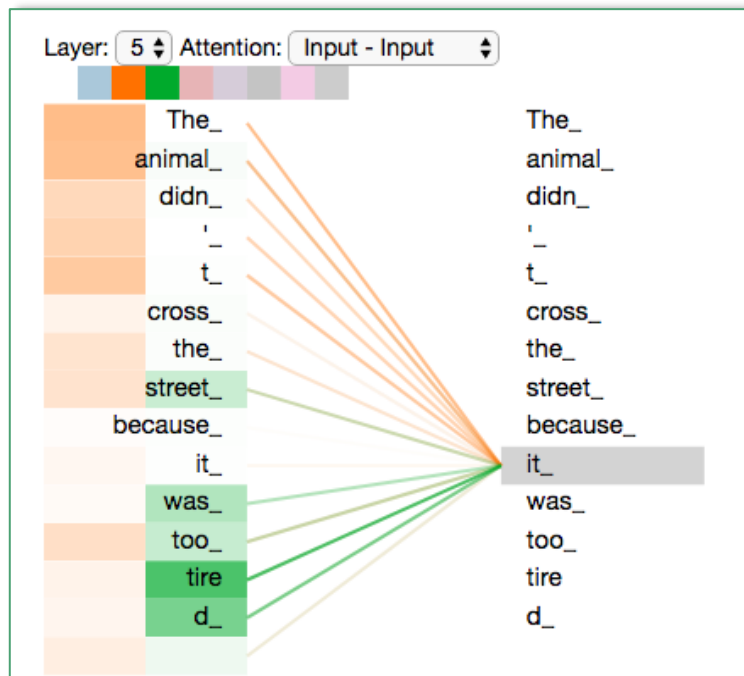
Encoder Transformer - BERT

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking Machines



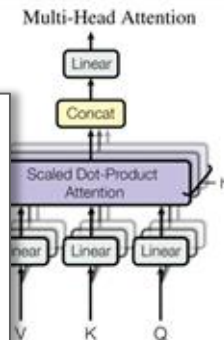
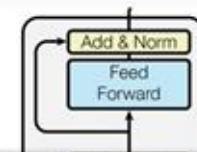
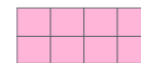
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



W^O



Z



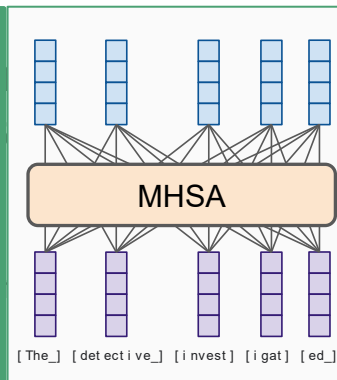
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

Encoder Transformer - BERT

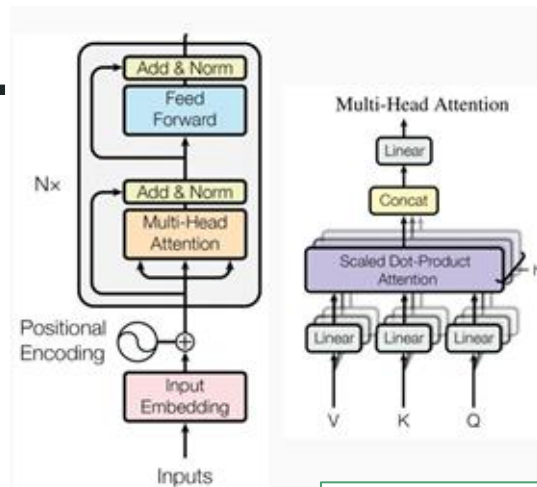
- Multi-headed self attention
 - Context modeling

Each token can "look around" the whole input, and decide how to **update its representation** based on what it sees.

The input sequence is used to create queries, keys, and values



From: Lucas Beyer [...]



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

$$= Z$$

Thinking Machines

Calculating attention separately in eight different attention heads

ATTENTION HEAD #0

Z_0

ATTENTION HEAD #1

Z_1

...

ATTENTION HEAD #7

Z_7

<https://jalamar.github.io/illustrated-transformer/>

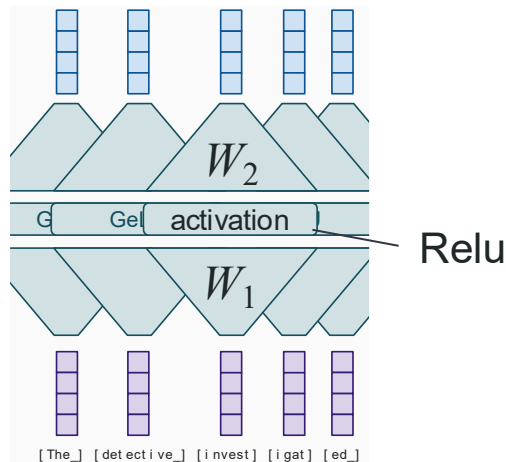
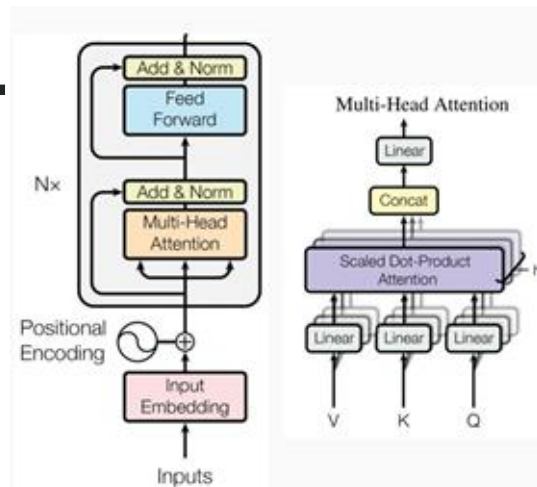
<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Each input vector is multiplied with two more matrices via a fully connected hidden layer (e.g., ~ dim = 2048)



<https://jalammar.github.io/illustrated-transformer/>

<https://arxiv.org/abs/1706.03762>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

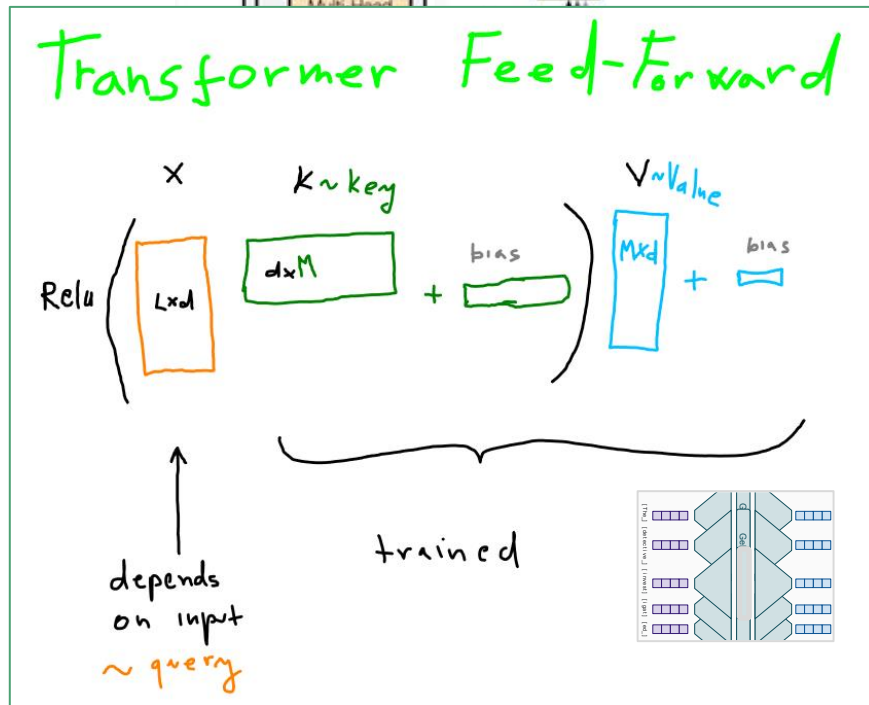
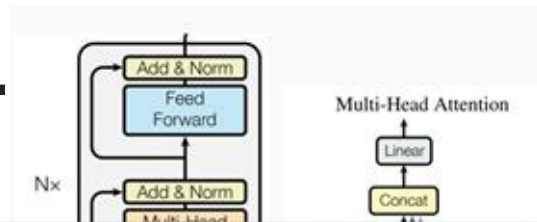
Each input vector is multiplied with two more matrices via a fully connected hidden layer (e.g., ~ dim = 2048)

~ maps **input** to **keys** + maps to **values**

~ act like triggers of syntactic > semantic patterns

Transformer Feed-Forward Layers Are Key-Value Memories. Geva & al EMNLP 2021.

(<https://aclanthology.org/2021.emnlp-main.446.pdf>)



From: <https://vaclavkosar.com/ml/Feed-Forward-Self-Attention-Key-Value-Memory>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

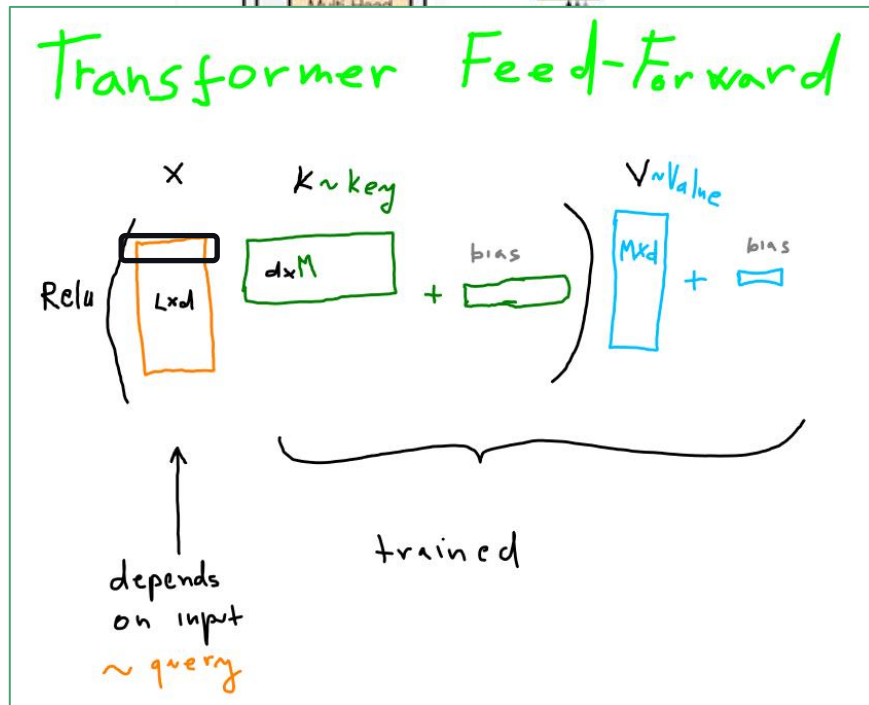
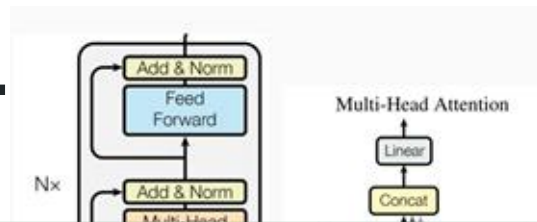
Each input vector is multiplied with two more matrices via a fully connected hidden layer (e.g., ~ dim = 2048)

~ maps **input** to **keys** + maps to **values**

~ act like triggers of syntactic > semantic patterns

Transformer Feed-Forward Layers Are Key-Value Memories. Geva & al EMNLP 2021.

(<https://aclanthology.org/2021.emnlp-main.446.pdf>)

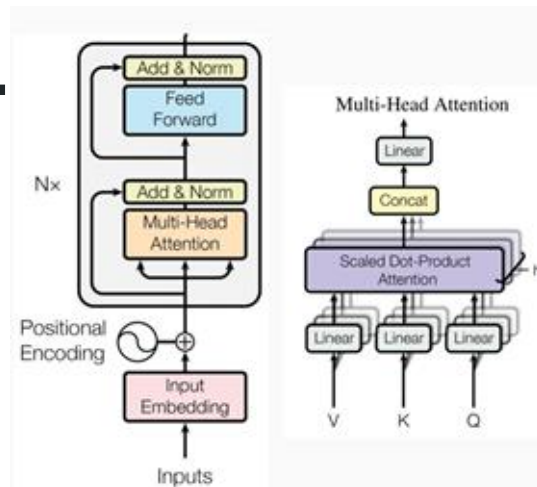


From: <https://vaclavkosar.com/ml/Feed-Forward-Self-Attention-Key-Value-Memory>

Encoder Transformer - BERT

- Multi-headed self attention
 - Context modeling
- Feed-forward layers
 - Non-linear hierarchical features
- Layer norm and residuals
 - Healthy training
- Positional embeddings
 - Relative positioning learning

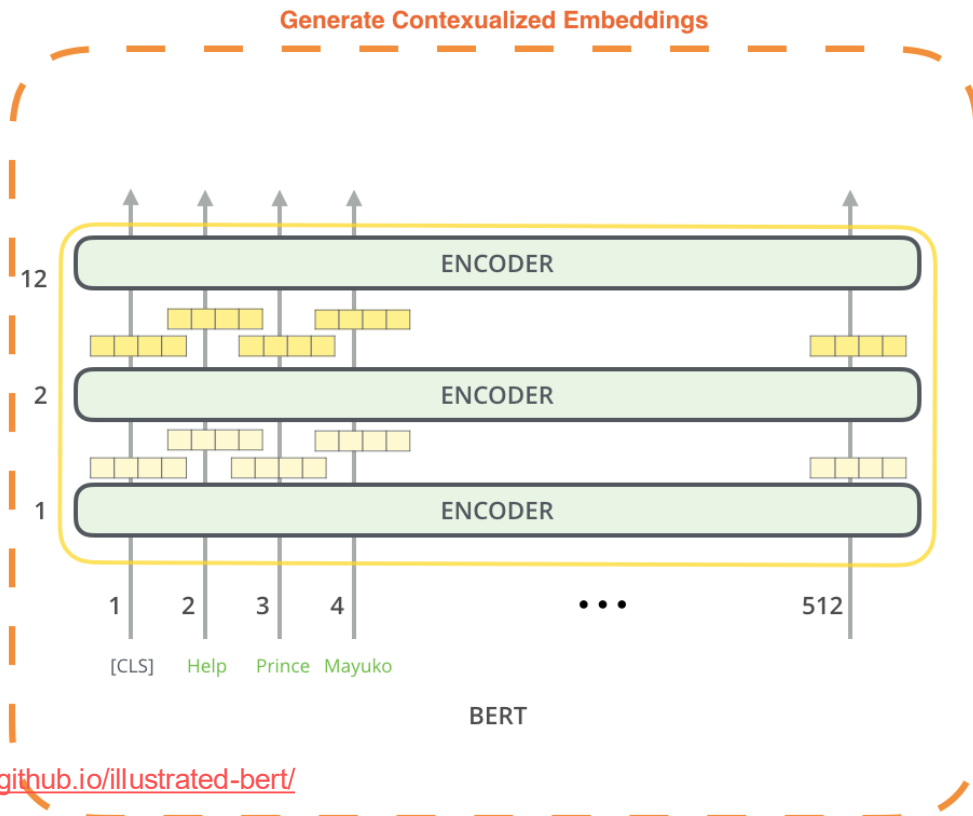
From: Lucas Beyer [...]



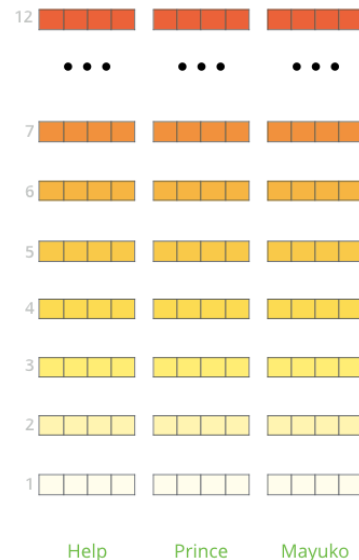
Encoder Transformers: Sequence Transformations

Learn
how to
convert
an input
sentence

Several
layers



The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

Masked Input to BERT (training/inference)

Sentence = "I am a policewoman"

Can be tokenized as:

- ['[CLS]', 'I', '[MASK]¹', 'a', 'police', '##woman', '[SEP]']

Things to remember:

1. Some tokens are **MASKED**² (to avoid words seeing themselves)
2. BERT is trained to predict the transformed token embeddings, but special tokens are used; e.g., [CLS], as a token to refer to the whole sentence

Transformers: Last Layer and Word Prediction

- BERT is trained to output the transformed vectors at inference time
- But for training we need to compute a gradient using the training data... we need to output a word, similarly as it is done for decoders (back to decoders later)

Which word in our vocabulary
is associated with this index?

am

Get the index of the cell
with the highest value
(**argmax**)

5

log_probs



Softmax

logits



Linear

Decoder stack output



Pre-training and Fine-tuning in Encoders: BERT

"Self-supervised"

1 - **Semi-supervised** training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

Model:



Dataset:



Objective:

Predict the masked word
(language modeling)

2 - **Supervised** training on a specific task with a labeled dataset.

Supervised Learning Step

Model:
(pre-trained
in step #1)



Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam

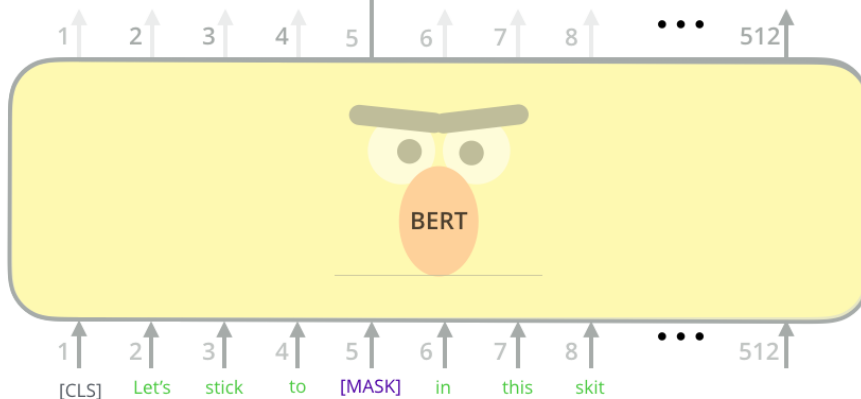
BERT pre-training

Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzyva

FFNN + Softmax



Randomly mask
15% of tokens

Input

[CLS] Let's stick to improvisation in this skit

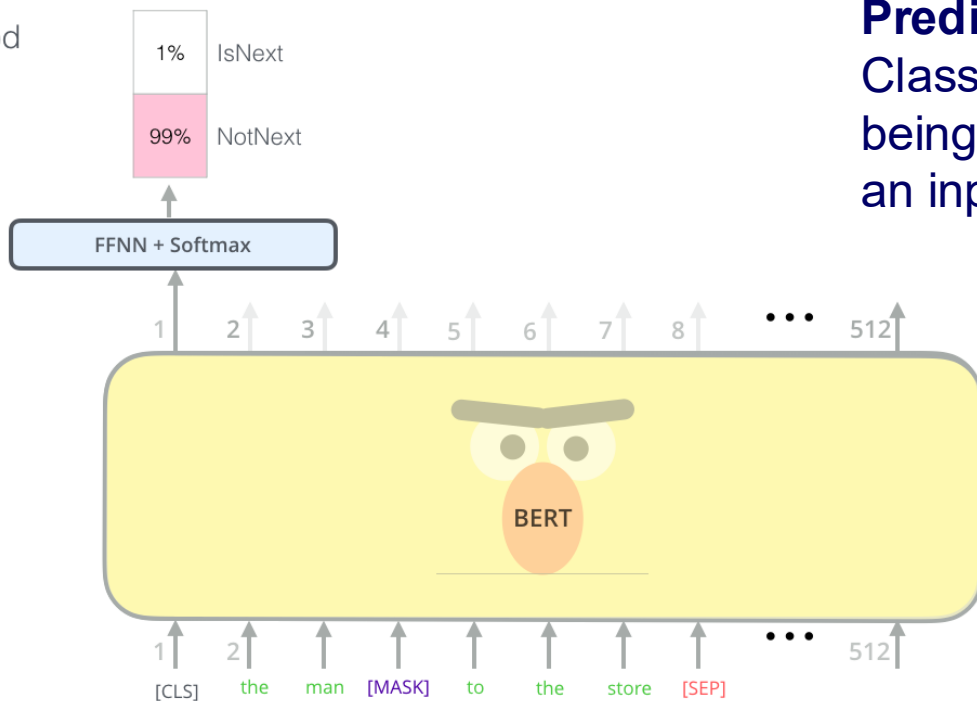
Predict masked words
Cloze

Predict next sentence
Classify a sentence as
being the continuation of
an input sentence or not

Source: <http://jalammar.github.io/illustrated-bert/>

BERT pre-training

Predict likelihood
that sentence B
belongs after
sentence A



Predict masked words
Cloze

Predict next sentence
Classify a sentence as
being the continuation of
an input sentence or not

Masking Strategy

It's a matter of tradeoff:

- Too little masking: Too expensive to train
- Too much masking: Not enough context

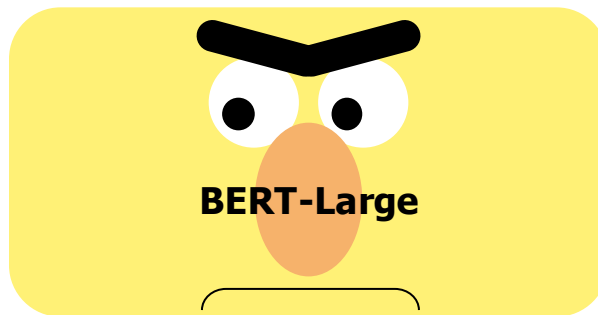
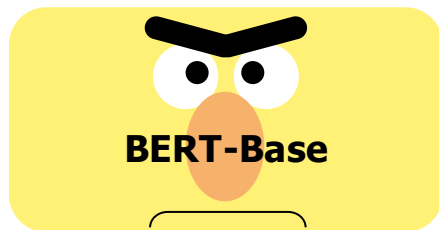
Suggested strategy (from the original paper):

- 80% of the time: mask the word with the [MASK] token
*I play with my cat -> I play with my **[MASK]***
- 10% of the time: replace the word with a random one
*I play with my cat -> I play with my **sugar***
- 10% of the time: keep the sentence as is
*I play with my cat -> I play with my **cat***

Pre-trained BERT Models

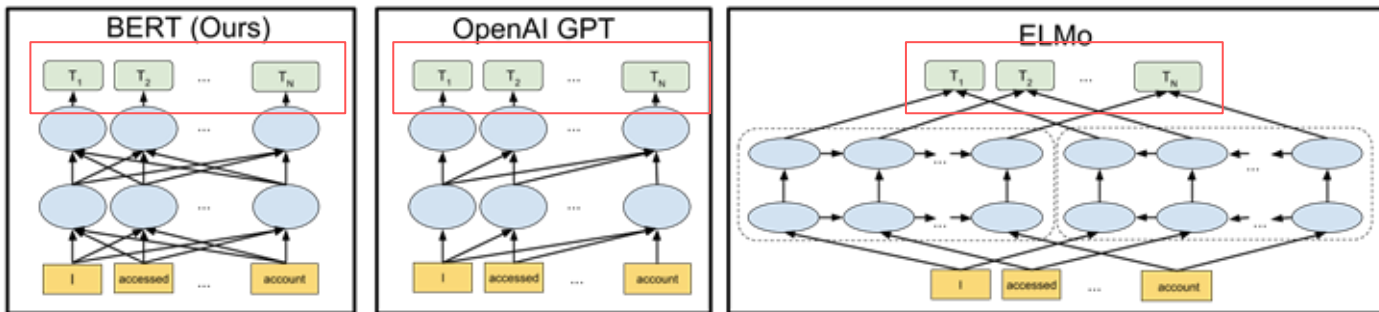
- **Data:** Wikipedia (2.5B words) + BookCorpus (800M words)
- **Batch Size:** 131,072 words (1024 sequences * 128 length or 256 sequences * 512 length)
- **Training Time:** 1M steps (~40 epochs)
- **Optimizer:** AdamW, 1e-4 learning rate, linear decay

12-layer
768-hidden
12-head



24-layer
1024-hidden
16-head

NLU for Downstream Tasks



- BERT et al. (encoders, especially), have been very effective to do **transfer learning**
- Easy ML:
 - Say you have to do sentiment analysis, you can download Google BERT pre-trained model, you get the embedding for your sentences (something you can do pretty quickly) and then you use a simple logistic classifier to learn the sentiment.
- You can also fine-tune BERT on a specific task, it's a good tradeoff between training from scratch and just used the pre-trained representations.

Pre-training vs Fine-Tuning vs Adapting in BERT

Use Pre-trained Embeddings:

- Extract embeddings from one of the layers and use them (like you do in word2vec)
 - We may see this in the colab notebook

Fine-Tune Embeddings:

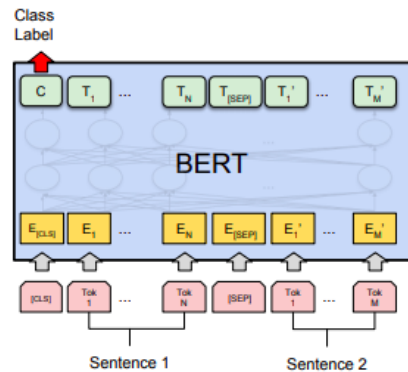
- Fine-tune all or some layers on a downstream task (e.g., classification)
 - This is generally affordable

Adapt Embeddings:

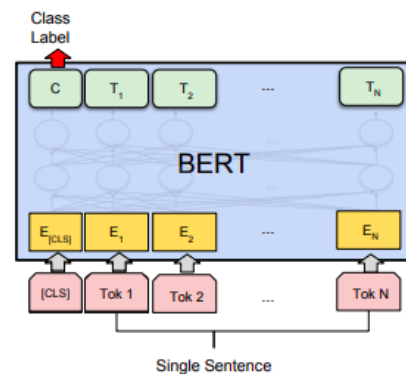
- Continue training based on the masked language modelling task on domain data

BERT architectures for downstream tasks

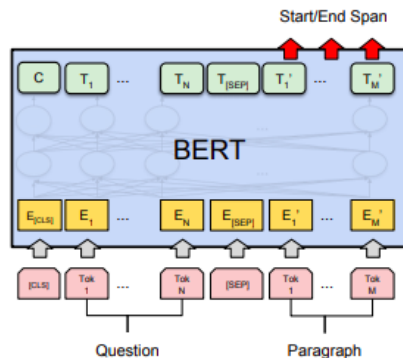
(task-specific fine-tuning)



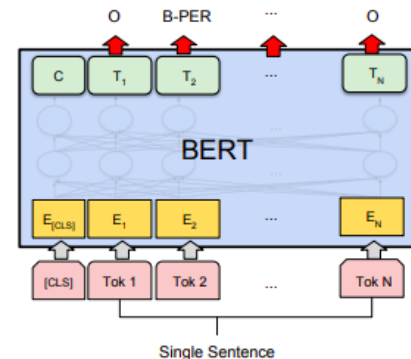
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



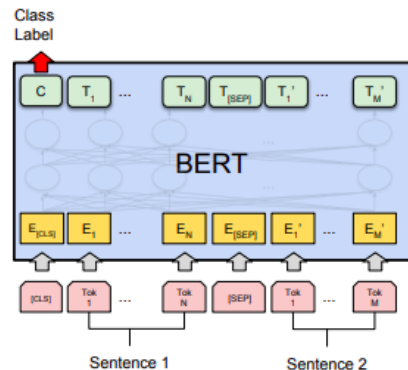
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Other details on fine-tuning strategies here:
<https://towardsdatascience.com/bert-3d1bf880386a>

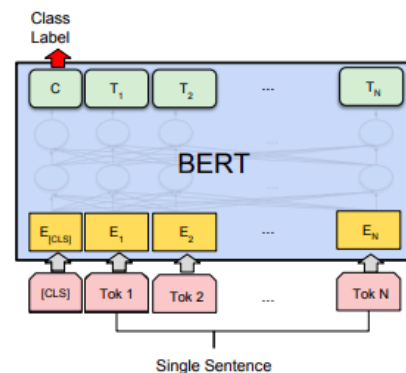
Source: <http://jalamar.github.io/illustrated-bert/>

BERT architectures for downstream tasks

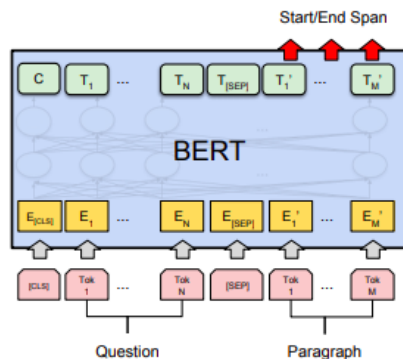
(task-specific fine-tuning)



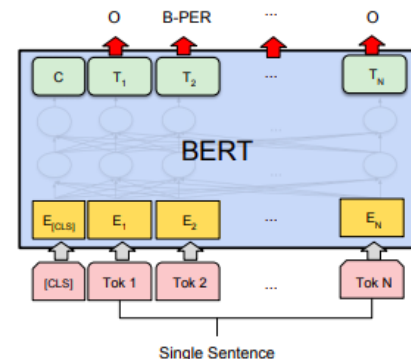
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



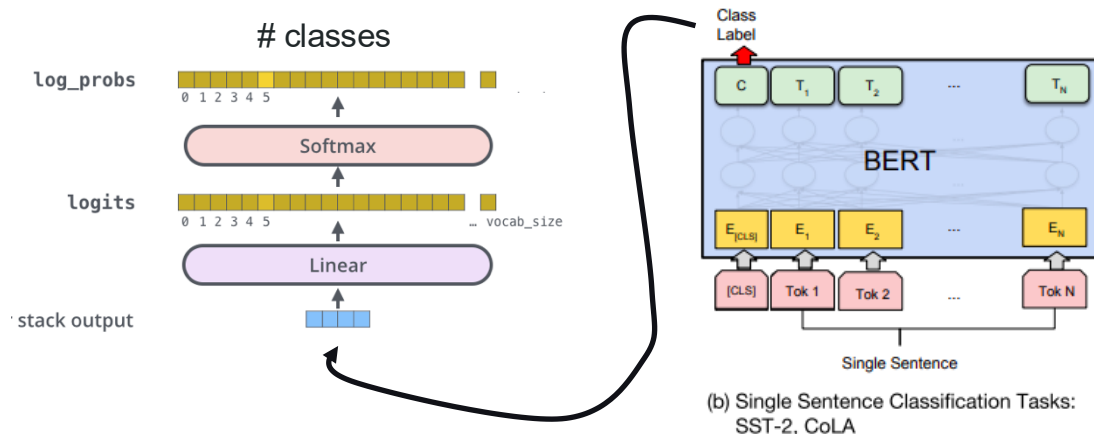
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Other details on fine-tuning strategies here:
<https://towardsdatascience.com/bert-3d1bf880386a>

Source: <http://jalamar.github.io/illustrated-bert/>

BERT architectures for data enrichment

Sentence classification



Train a classifier for an input text

- Classification using the output of the [CLS] token
- Fine-tuning
 - Linear layer only
 - BERT weights + linear layer
- Examples:
 - Sentiment with 3 classes
POS | NEG | NEU

Some useful variants:

- SBERT, BERT revised for sentence representation

Other details on fine-tuning strategies here:

<https://towardsdatascience.com/bert-3d1bf880386a>

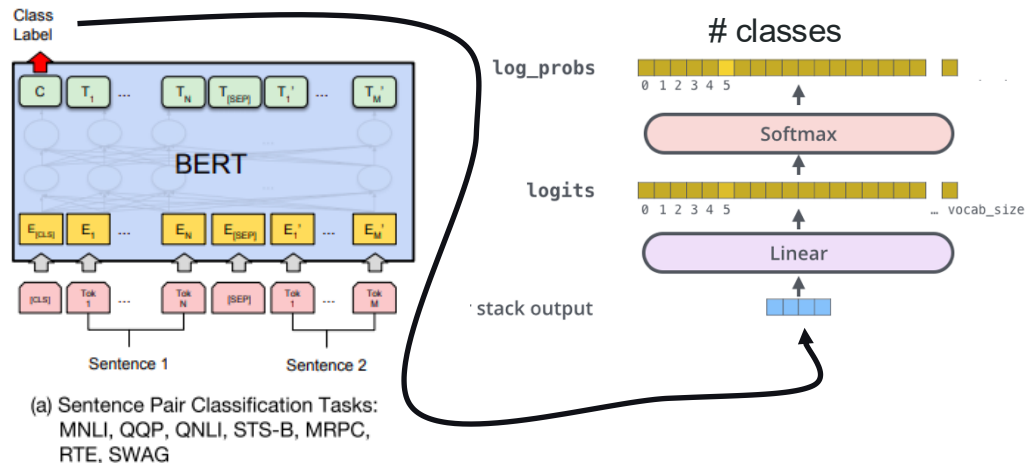
BERT architectures for data enrichment

Sentence pair classification

Train a classifier for an input text

- Classification using the output of the [CLS] token
- Fine-tuning
 - Linear layer only
 - BERT weights + linear layer
- Examples:
 - The **cross-encoder** pattern
 - Entity matching with 2 classes: Y | N

[CLS] entity1_descr [SEP] entity2_descr



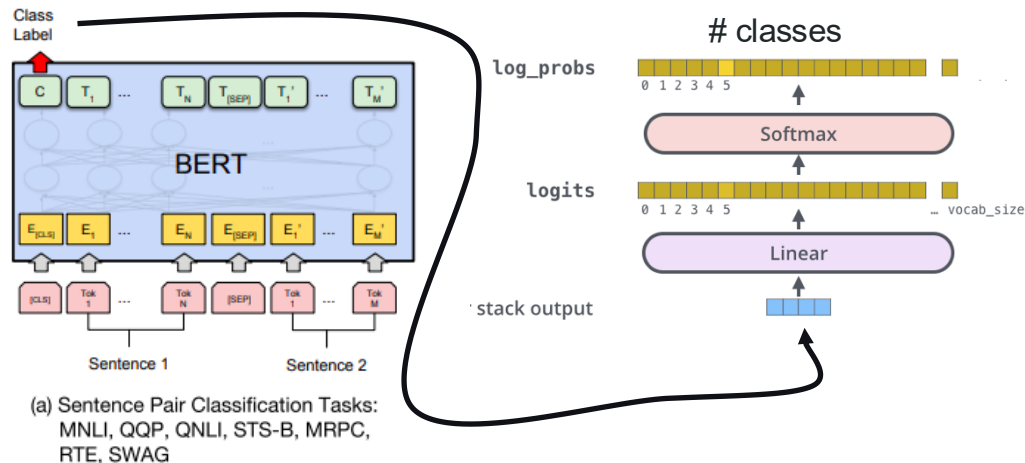
BERT architectures for data enrichment

Sentence pair classification

Train a classifier for an input text

- Classification using the output of the [CLS] token
- Fine-tuning
 - Linear layer only
 - BERT weights + linear layer
- Examples:
 - The **cross-encoder** pattern
 - Entity matching with 2 classes: Y | N

[CLS] entity1_descr [SEP] entity2_descr



Examples of downstream applications:

- Entity matching for relational data: DITTO
 - entity1_descr =

[COL] attr₁ [VAL] val₁ ... [COL] attr_k [VAL] val_k,

- Generalized cross-encoder: UNICORN
 - <https://dl.acm.org/doi/10.1145/3665252.3665263>

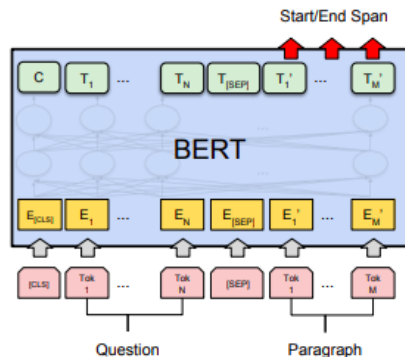
BERT architectures for data enrichment

Question answering

Detect beginning and end of a sub-paragraph containing the answer to a question

Linear layer mapping to appropriate classes (as before)

+



(c) Question Answering Tasks: SQuAD v1.1

Is Jurassic Park appropriate for a 7 year old?

Overall, I would say it is not suitable for kids under the age of 15. The plot and overall flow of the storyline is great, the characters have depth, and the dinosaurs look cool and quite realistic. There is minimal swearing, only one "f--k" word in the whole movie. I wouldn't consider this to be of any concern. 12 ju 2015

Quora
<https://www.quora.com/Is-Jurassic-World-suitable-for-...>

Is Jurassic World suitable for kids? - Quora



Quora

Is Jurassic World OK for a 7-year-old?

All related (35)

Sort Recommended

Emmy Wagner
Studied at The High School Experience (Graduated 2021) · 4y

Yes, my 5 year old watches it all of the time and is one of her favorite movies!!

But then again, it just depends on the parent + child.

If the parent thinks it is okay, then go for it! Your child will tell you if they are scared and wants you to turn it off.

Upvote · 1 · Share · Comment · Refresh

Dustin Nguyen
Watched the films and read the novel. · Author has 3.9K answers and 10M answer views · 4y

Well, I wouldn't say keep this away from kids because it's okay of a film despite some moments of violence that might be a tad jump raising.

...compared to the outright horror that was the original film, Jurassic World is pretty tame.

Upvote · 1 · Share · Comment · Refresh

QueenOfDragons
Personality type is INFJ · Author has 2K answers and 2.9M answer views · 2y

No.

There are heaps of scenes where people are killed/eaten by dinosaurs and most of it is quite graphic. The first movie is a fair bit tamer than the second one.

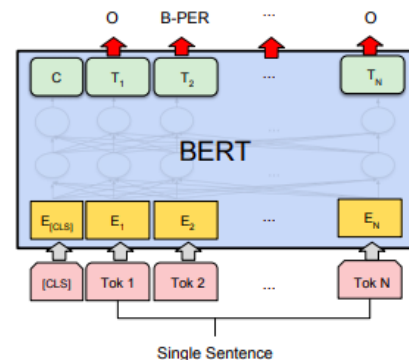
BERT architectures for data enrichment

Sequence tagging, e.g., Named Entity Recognition

Other details on fine-tuning strategies here:
<https://towardsdatascience.com/bert-3d1bf880386a>

Example: extracting entities from texts

... more details on this in the dedicated lesson



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Source: <http://jalamar.github.io/illustrated-bert/>

Encoders for Data Enrichment: Recap

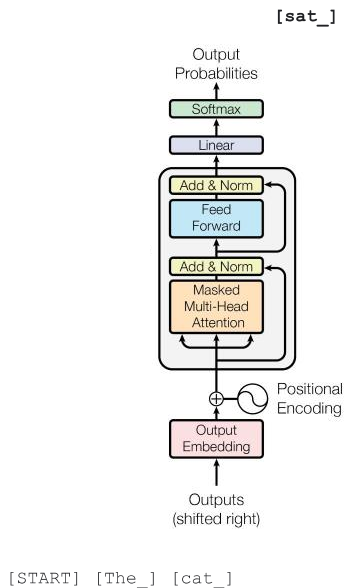
- Generate representations from the input context
 - Contextualized token representations at different layers
 - Sentence representation from [CLS]
 - Sentence comparison representation from [CLS] + [SEP]
- Classifiers applied to representations for enrichment
 - Typically: a linear layer producing score for each class + softmax
 - Training options:
 - Train only the classifier
 - Train all or some weights from the transformers
 - Use a Parameter Efficient Fine-Tuning (PFET) option, e.g., Adapters

Transformers

Architectures and key components

Decoders: Natural Language Generation

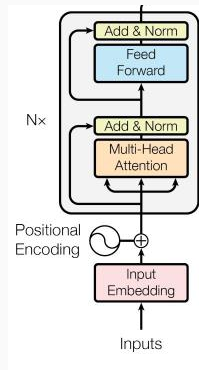
Decoder-only GPT



Transformer image source: "Attention Is All You Need" paper

Encoder-only BERT

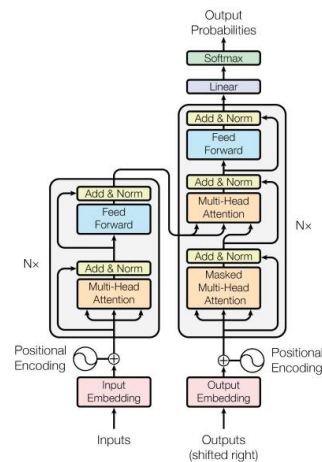
[*] [*] [sat_] [*] [the_] [*]



[The_] [cat_] [MASK] [on_] [MASK] [mat_]

Enc-Dec T5

Das ist gut.
A storm in Attala caused 6 victims.
This is not toxic.



Translate EN-DE: This is good.
Summarize: state authorities dispatched...
Is this toxic: You look beautiful today!

Decoder-only GPT

The diagram illustrates the GPT decoder-only architecture. At the bottom, the input sequence is `[START] [The_] [cat_]` (where the backtick indicates a space). This sequence is processed by an Output Embedding block. A Positional Encoding is added to the output of the embedding block. The resulting sequence then passes through a stack of identical decoder blocks. Each block contains a Masked Multi-Head Attention layer, followed by an Add & Norm layer (residual connection), a Feed Forward network, and another Add & Norm layer (residual connection). The output of the final decoder block is passed through a Linear layer and a Softmax layer to produce the final Output Probabilities, which are shown as [sat_]`.`

[START] [The_] [cat_]`

Transformer image source: "Attention Is All You Need" paper

Transformer image source: "Attention Is All You Need" paper

Encoder-only

BERT

Same as the module used to train BERT with the masked language modeling tasks

[*] [*] [sat_] [*] [the_] [*]

=

Positional Encoding

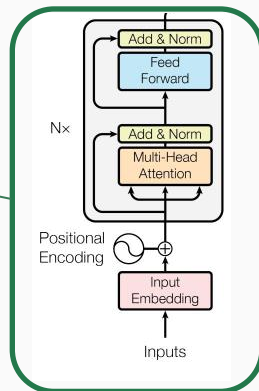
Input Embedding

Inputs

[The_] [cat_] [MASK] [on_] [MASK] [mat_]

Same as the module used to train BERT with the masked language modeling tasks

[*] [*] [sat] [*] [the] [*]



[The] [cat] [MASK] [on] [MASK] [mat]

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.

The diagram illustrates a sequence-to-sequence model architecture, likely a Transformer-based encoder-decoder. It consists of two main parts: an encoder (left) and a decoder (right).

Encoder (Left):

- Inputs:** A sequence of inputs is fed into an **Input Embedding** block.
- Positional Encoding:** A sine/cosine wave icon represents positional encoding, which is added to the input embeddings.
- Encoder Block (Nx):** The encoder consists of $N \times$ identical blocks. Each block contains:
 - Multi-Head Attention:** Processes the input embeddings.
 - Add & Norm:** Residual connection and layer normalization.
 - Feed Forward:** A feed-forward network.
 - Add & Norm:** Another residual connection and layer normalization.

Decoder (Right):

- Outputs (shifted right):** The decoder takes shifted right outputs as input.
- Positional Encoding:** A sine/cosine wave icon represents positional encoding, which is added to the output embeddings.
- Decoder Block (Nx):** The decoder consists of $N \times$ identical blocks. Each block contains:
 - Masked Multi-Head Attention:** Processes the shifted right outputs.
 - Add & Norm:** Residual connection and layer normalization.
 - Multi-Head Attention:** Receives input from the encoder's output.
 - Add & Norm:** Residual connection and layer normalization.
 - Feed Forward:** A feed-forward network.
 - Add & Norm:** Residual connection and layer normalization.

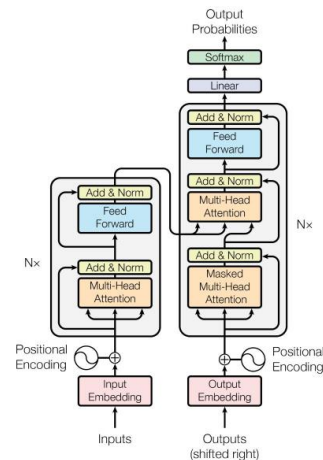
Output:

- The final output from the decoder is passed through a **Linear** layer and a **Softmax** layer to produce the **Output Probabilities**.

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.

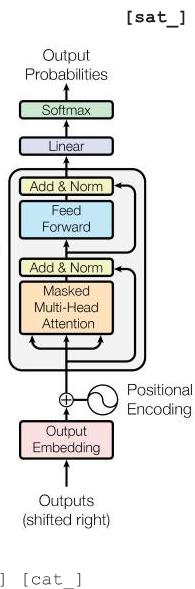


Translate EN-DE: This is good.

Summarize: state authorities dispatched...

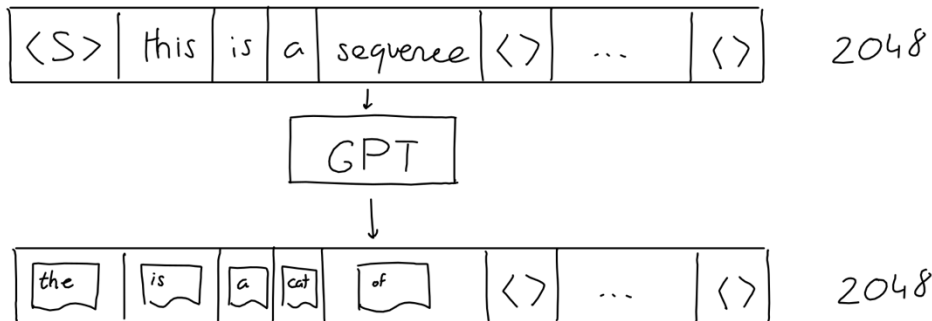
Is this toxic: You look beautiful today!

Decoder-only GPT



Transformer image source: "Attention Is All You Need" paper

Autoregressive Pre-trained Language Models



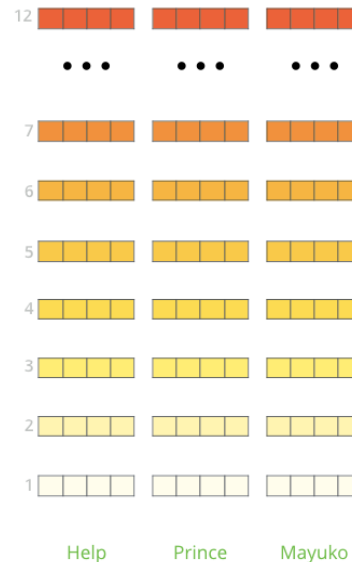
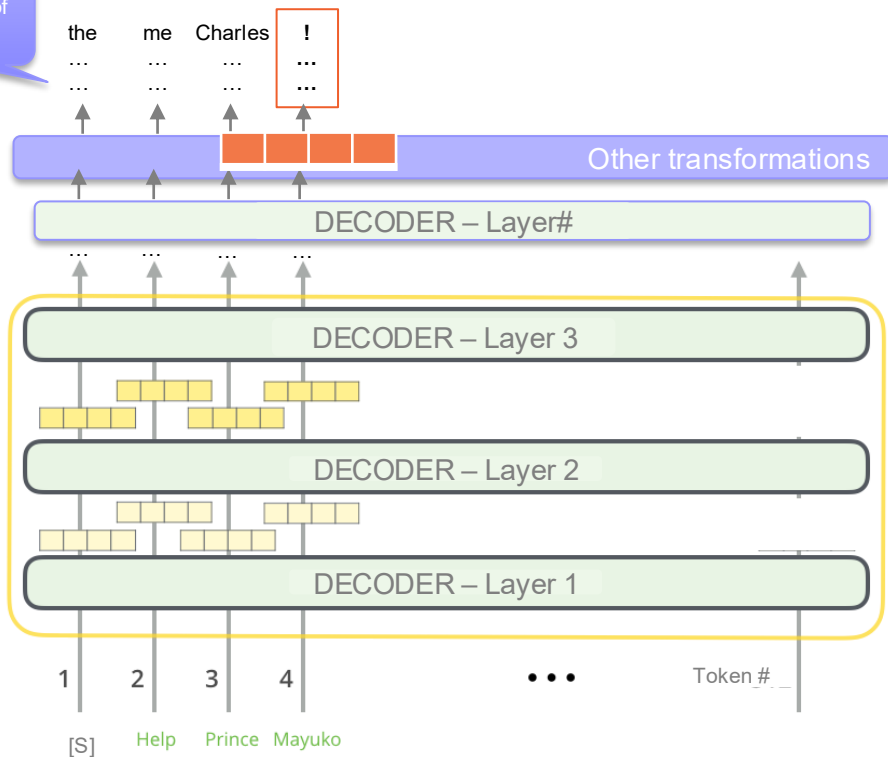
1. The input sequence is actually fixed to 2048 words (for GPT-3). We can still pass short sequences as input: we simply fill all extra positions with "empty" values.
2. The output is not a single guess, it's a sequence (length 2048) of guesses (a probability for each likely word). One for each 'next' position in the sequence. But when generating text, we typically only look at the guess for the last word of the sequence.

https://dugas.ch/artificial_curiosity/GPT_architecture.html

3. **Autoregressive:** at each time t we pick up what was generated at time $t-1$ and feed it as input to the network

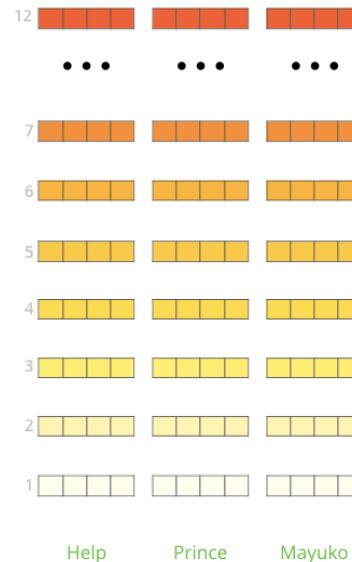
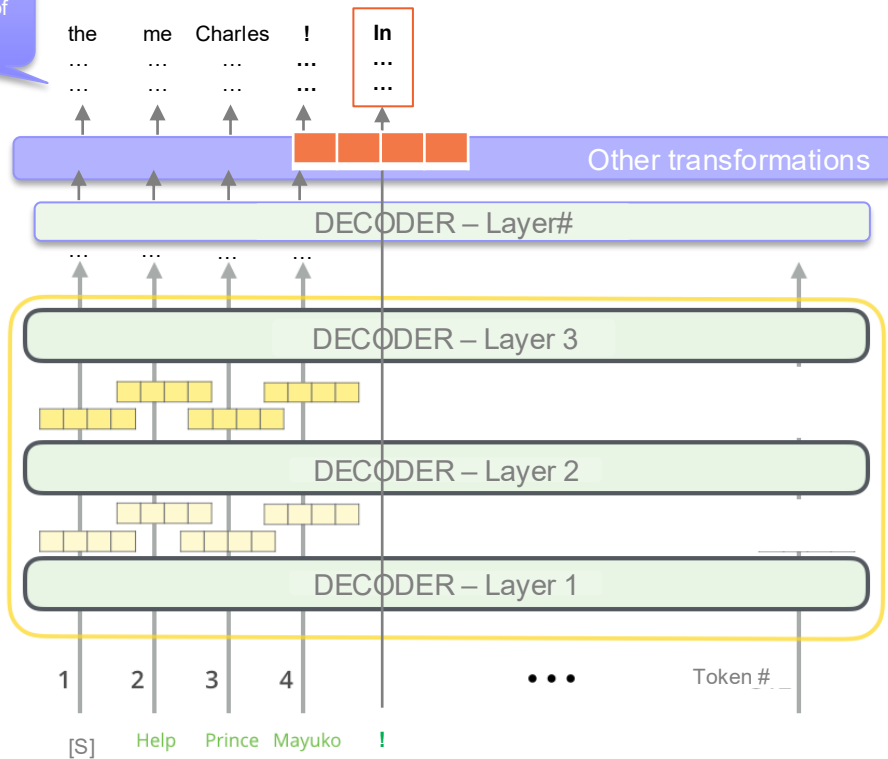
Generative LLMs: Auto-regressive Models

Probability of each token



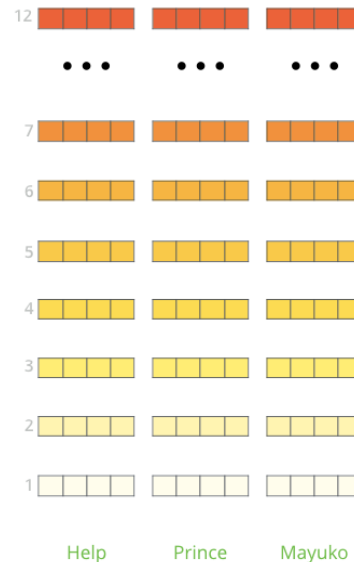
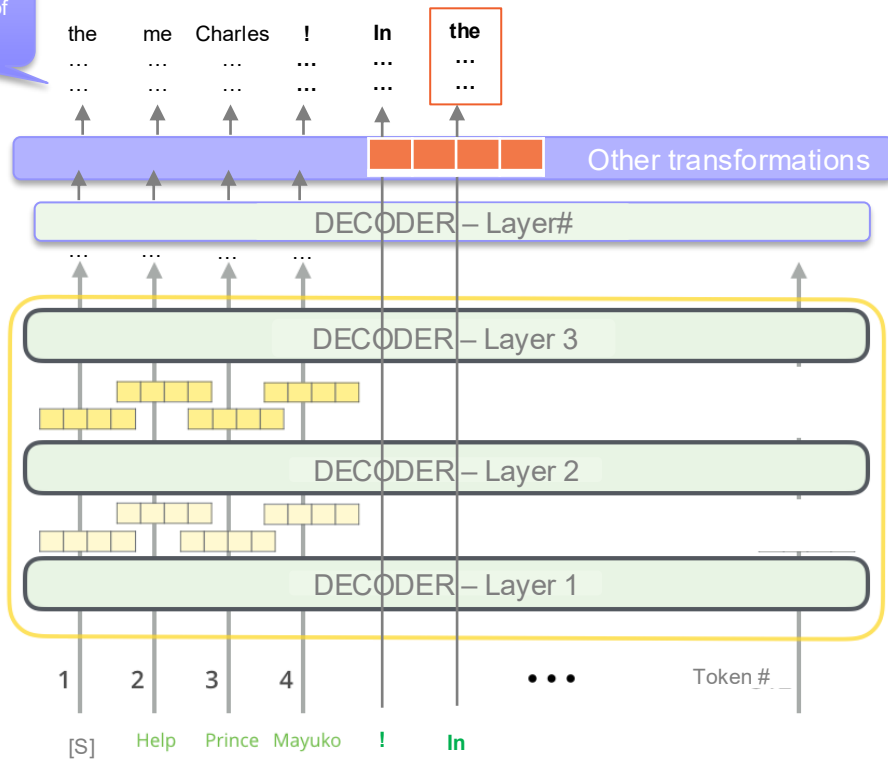
Generative LLMs: Auto-regressive Models

Probability of each token



Generative LLMs: Auto-regressive Models

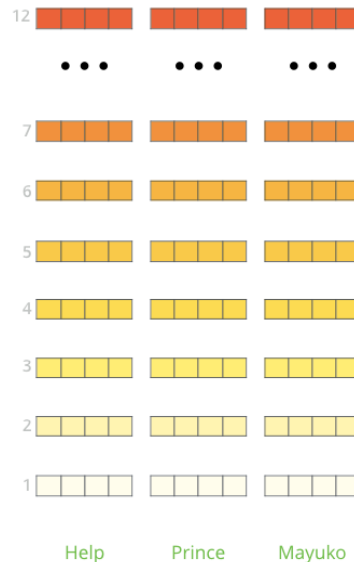
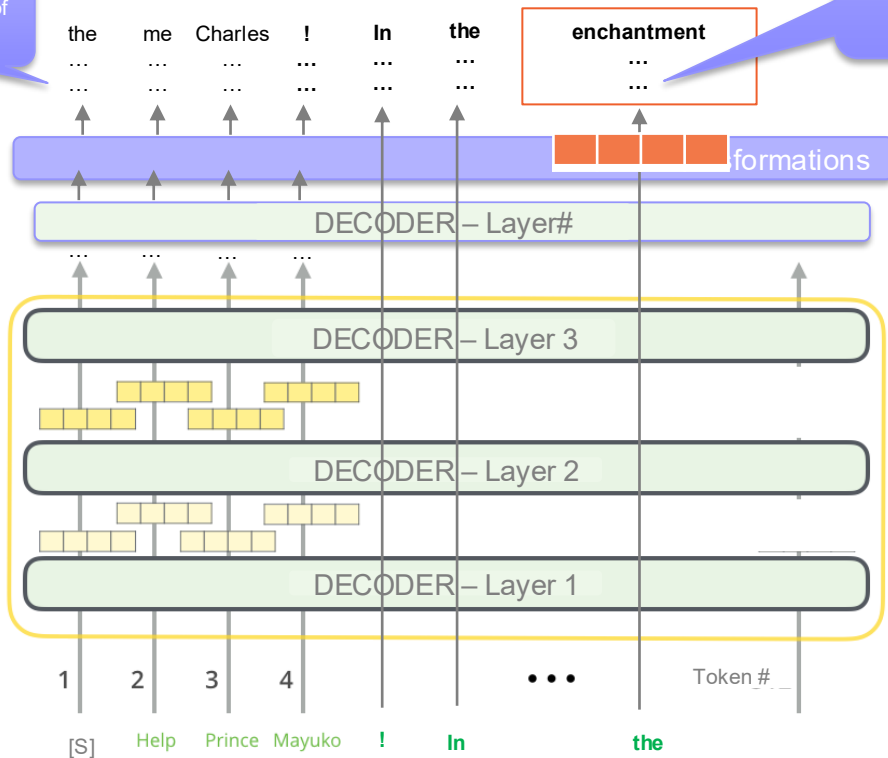
Probability of each token



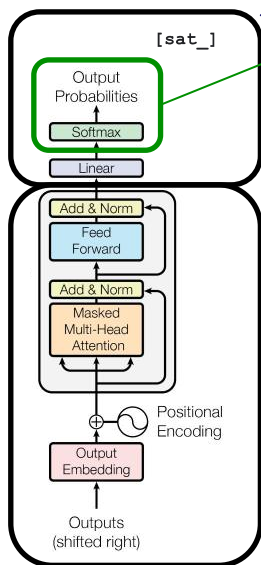
Generative LLMs: Auto-regressive Models

Probability of each token

Instead of picking the most likely, we can pick some other likely token (see: temperature)



Decoder-only GPT



[START] [The_] [cat_]

Transformer image source: "Attention Is All You Need" paper

SoftMax computed with logits from the linear layer,
same as in the module used to train BERT

Several variants of this module have
been proposed

Some issues:

- Picking the most probable token may make generation deterministic and little creative
 - >> **sampling tokens from the probability distribution**
 - >> **different sampling strategies are possible** ... we'll skip details in next slides, but it's worth noting that:
- Determinism vs. creativity can be controlled by an hyperparameter >> **temperature**

Decoding Sampling / Effect of Temperature #1

z_i and z_j are logits (output from linear layer)

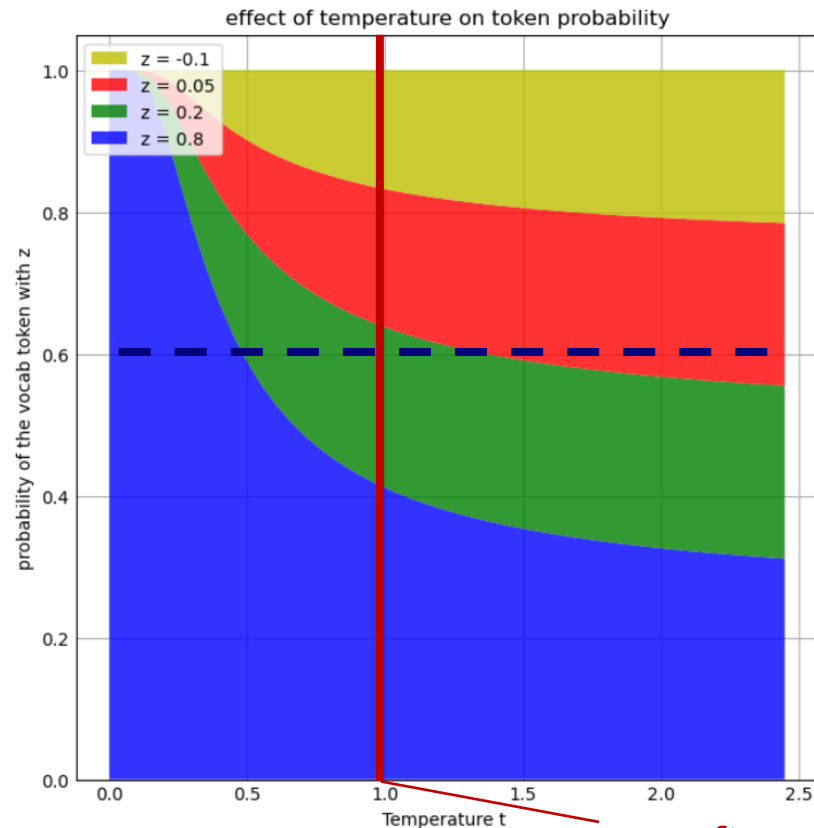
$$p_i(T) = \frac{e^{\frac{z_i}{T}}}{\sum_{j=1}^K e^{\frac{z_j}{T}}} \quad \text{Softmax with temperature } T$$

Usually applied to subset of tokens

- Top- k : list of k tokens with higher probability
- Top- p : list of tokens order by probabilities that sum up to p

Actual sampling: weighted random choice

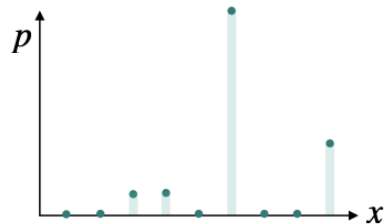
- Pick a random number $\in [0,1)$, i.e., a horizontal line in the square - - -
- Select the token intersected by the segment
- The probability of hitting a token will be proportional to the length of its segment



= softmax

Decoding Sampling / Effect of Temperature #2

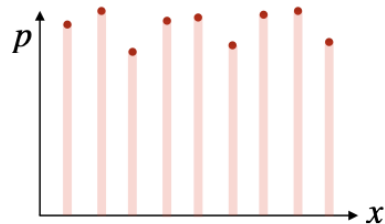
Softmax with temperature T^*



$T \ll 1$

$$p_i = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^n \exp\left(\frac{x_j}{T}\right)}$$

$T \gg 1$



$T < 1$ sharpen differences and increase the probability of picking tokens with high probabilities

- more deterministic behavior
- deterministic at $T = 0$

$T > 1$ flatten differences and increase the probability of picking tokens with lower probabilities

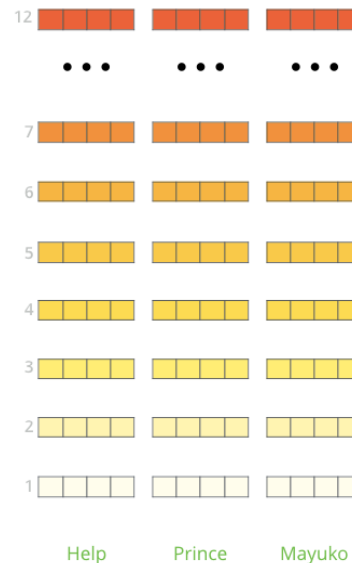
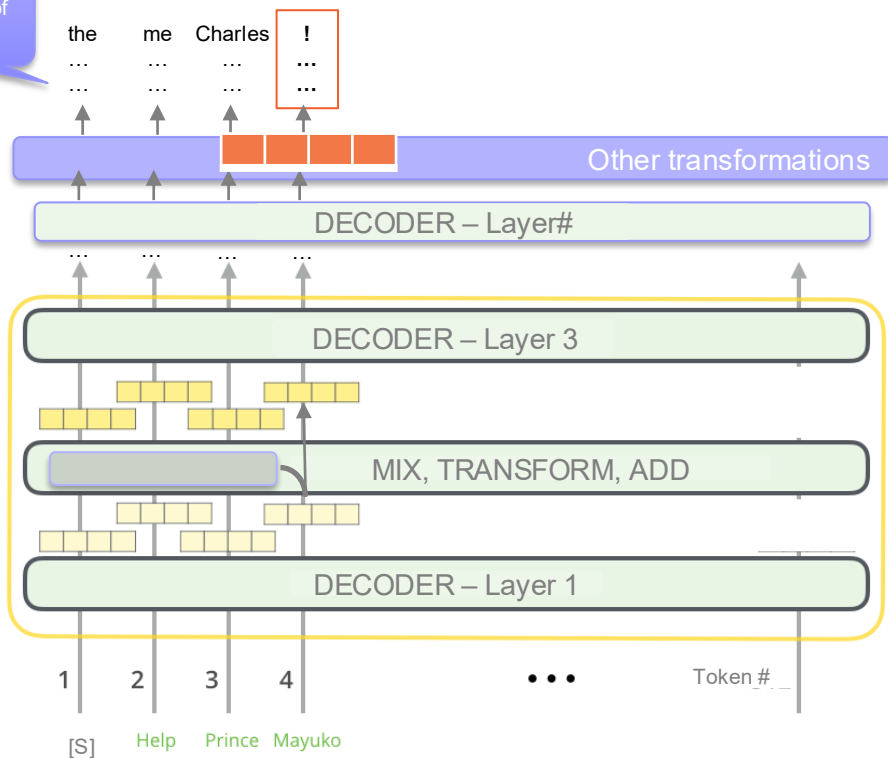
- more creative behavior

Source: <https://github.com/afshinea/stanford-cme-295-transformers-large-language-models/blob/main/en/cheatsheet-transformers-large-language-models.pdf>

* Remark: in this formulation logits are indicated by x_i and x_j

LLMs: Transformers and Attention

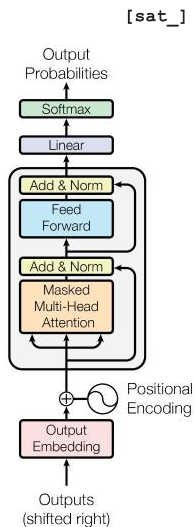
Probability of each token



Variations on the Transformers' Architecture

- Mixture of Experts (MoE): replace a big FFNN in the FF Layer with smaller FFNNs
 - Faster inference
- Variations of attention to deal with long contexts
- ... more variations exist

Decoder-only GPT

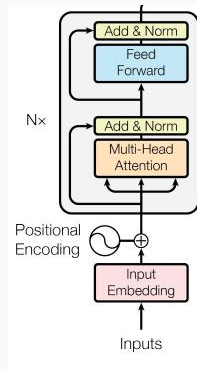


[START] [The_] [cat_] [sat_] [the_] [MASK]

Transformer image source: "Attention Is All You Need" paper

Encoder-only BERT

[*] [*] [sat_] [*] [the_] [*]



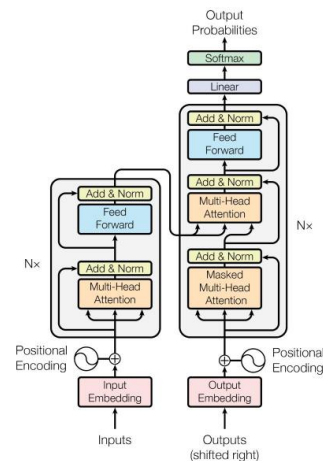
[The_] [cat_] [MASK] [on_] [MASK] [mat_] [sat_] [the_] [MASK]

Enc-Dec T5

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.



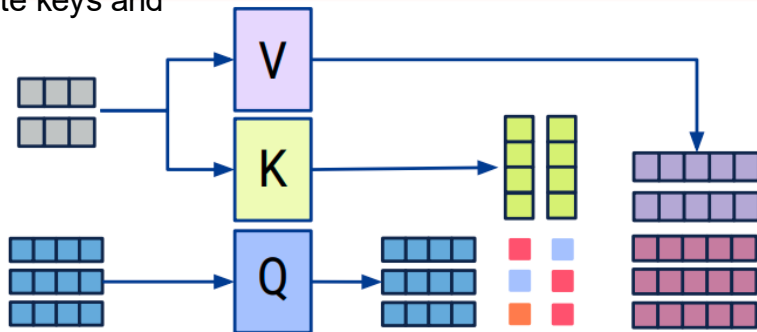
Translate EN-DE: This is good.

Summarize: state authorities dispatched...

Is this toxic: You look beautiful today!

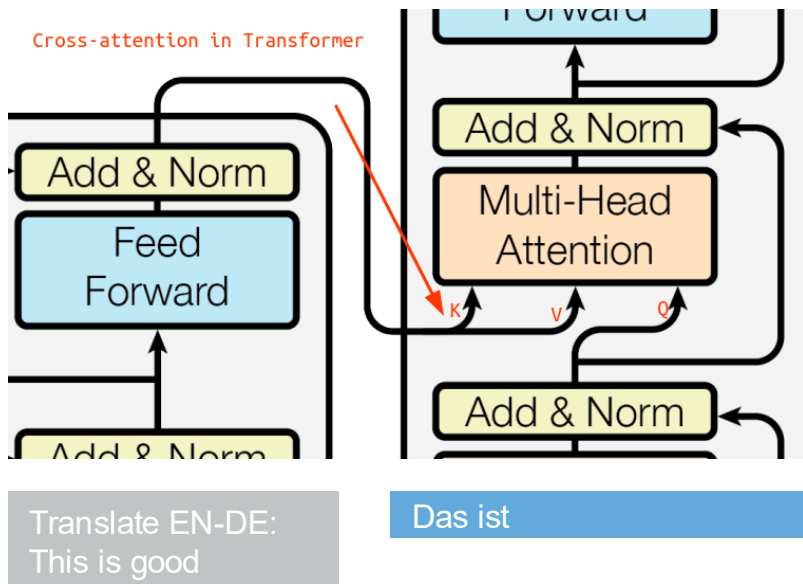
Mixing Enc-Dec Signals in Transformers: Cross Attention

Input sequence used to generate keys and values



Sequence generated so far used to generate queries

Attention scores

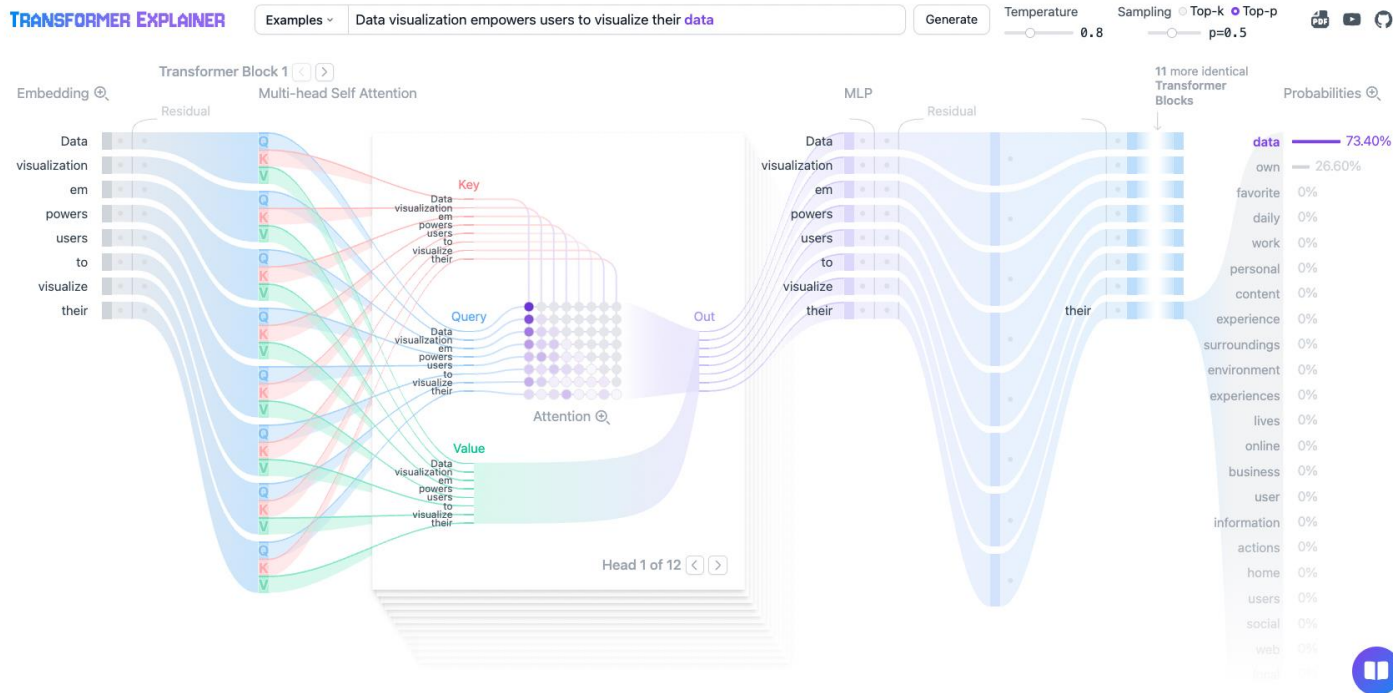


Source: <https://vaclavkosar.com/ml/cross-attention-in-transformer-architecture>

Decoders (and Encoder-decoders) for Data Enrichment

- Enrichment tasks must be transformed into a generative task
 - Naturally generative task
 - Translate
 - Sumarize
 - Answer questions
 - ...
 - Tasks typically interpreted as classification tasks reformulated as generative tasks
 - Text classification: label generation
 - Information extraction: generate extracted entities
 - ...

Visualization-based Explanation of Transformers



<https://poloclub.github.io/transformer-explainer/>

Transformers and LMs: Wrap-up

- Transformers: transform token representations across layers; update representations based on the context
- Different architectures
 - Encoders: extract features to feed additional ML layers
 - Decoders (and encoder-decoders): generate content
- Key mechanisms:
 - Multi-head attention
 - Feed forward network
 - ... but do not forget skip connections and existence of positional embeddings